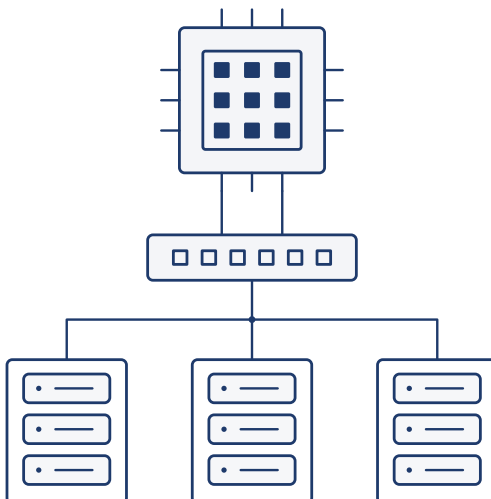


深入理解 AI Infra

量化分析与系统设计



李博杰 著

第 2 章排版样张 • 2026 年 9 月 10 日

目录

第 2 章 模型架构	1
2.1 计算图与前向执行	1
2.2 自回归生成的资源消耗计算	9
2.3 上下文状态表示与访问机制	12
2.4 条件计算与专家权重复用	23
2.5 网络形状与计算组织	26
2.6 整个模型的资源需求与比较	29
谬误与陷阱	39
本章练习与复现	40
模型矩阵查阅表	40
本章小结	46

第 2 章 模型架构

第一章从存储容量、运算量和数据读取量三个方面分析了一次模型执行，并借助硬件的关键数字估算时间下界。这样的估算可以迅速排除一些方案，却还留下一个问题：所代入的计算量和数据量从哪里来？“一个 8B 模型”只说明参数数量的大致范围，没有说明每个 token 使用哪些参数、上下文保存在什么地方，也没有说明不同工作之间的先后关系。

本章从一个 token 的数值表示开始，逐步分析它经过网络各层后，如何得到下一个 token 的分数，再计算这一过程所需的存储容量 M 、运算量 F 和读写量 R 。计算图用节点表示运算，用边表示运算之间传递的数据。先认识图上的对象，才能知道资源预算中的每个数字来自哪里。

这张计算图也提供了优化实现所需的信息。哪些权重在不同请求间保持不变，哪些状态会被后续生成反复使用，哪些结果只供紧接着的运算使用，都可以从模型结构中找到。后面的存储、编译和调度设计将利用这些数据在用途上的区别，决定哪些数据可以共享、哪些需要长期保存、哪些只需临时缓冲。

我们使用 Qwen3-8B 作为贯穿例子。这是一个约含 81.9 亿参数的语言模型，通过 36 层网络处理文本。先完整理解这一种模型，再改变上下文的保存方式、每个 token 使用的参数集合和层间连接，比较 Qwen3.6-35B-A3B（下文简称 Qwen3.6）、DeepSeek V4-Flash 与 Kimi K3 的资源需求。

统计运算量时，一次乘法加一次加法计为 2 FLOPs。存储量采用 GB 和 GiB：1 GB = 10^9 bytes，1 GiB = 2^{30} bytes。BF16 是每个数占 16 位的浮点表示，每个元素占 2 字节，因此将张量的元素数乘以 2，就得到它的存储大小。

2.1 计算图与前向执行

张量是按一个或多个维度排列的数值数组：向量只有一个维度，矩阵有两个维度。模型用向量表示每个 token 的特征，把多个位置的向量按行排成矩阵。理解模型资源需求，首先要把计算节点和数据依赖分开。一个节点执行矩阵乘法或其他变换，一条边表示后续节点需要前面的结果。节点的运算量决定计算需求，边上的张量决定传递的数据量，边的方向则表明哪些运算必须先后执行。理解模型的执行过程，需要同时考虑这三个方面。

本章的计算图先描述数学运算和数据依赖。实际执行时，框架会把这些运算交给设备程序；一个运算可能拆成多个程序，多个运算也可能合在一起执行。计算图中表示张量的一条边，只说明后续运算需要这份数据，不表示它一定要在 CPU 与 GPU 之间复制。第四章说明数据放在哪里，第五章再解释程序怎样提交、数据怎样传入传出。

2.1.1 序列依赖与因果计算图

要计算执行时间，先要知道哪些运算能够同时开始。考虑四个 token、三层网络，把位置画在横轴、层数画在纵轴。循环神经网络（RNN）把前一位置的状态传给后一位置，位置状态满足 $h_t = f(x_t, h_{t-1})$ 。其中 x_t 是当前输入， h_t 是处理到当前位置时保存的状态， f 是使用模型权重进行的变换。当前结果依赖前一位置，所以同一层按位置顺序执行。即使四个输入已经全部到达，第 t 个位置仍要等待第 $t-1$ 个位置。

Transformer 用注意力机制让一个位置读取其他位置的信息；“因果”表示当前位置只能使用自身及此前位置的信息。因果 Transformer 的依赖形式不同。第 l 层计算某个位置时，通过注意力读取上一层中该位置及其之前各位置的表示。只要输入 token 已知，并且上一层的相应表示已经完成，同一层不同位置的计算就不需要再等待本层前一个位置的输出。因果掩码用来规定各位置允许访问哪些信息，它屏蔽当前位置之后的信息。按

同层先完成前一位置

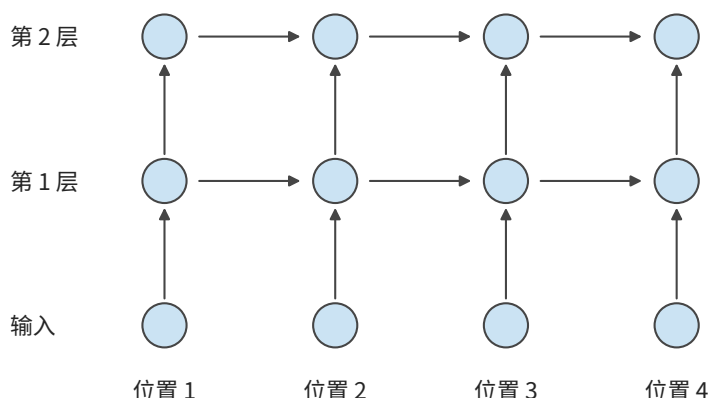


图 2-1 循环神经网络的依赖。相同颜色的圆点表示各位置的计算，箭头指向需要前一结果的节点；同层位置沿横向逐次推进。

查询与被查询位置排列，这张表呈三角形；它限制信息来源，却不要求所有已知位置逐个执行。网络深度方向的依赖仍然存在：下一层使用这一层的结果。

本层各位置读取上一层的因果前缀

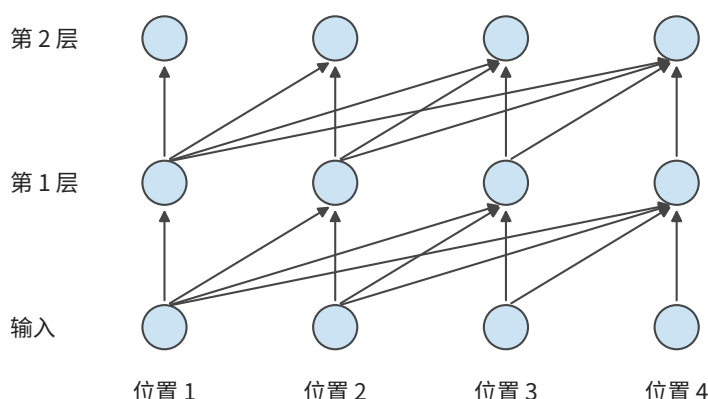


图 2-2 因果 Transformer 的层间依赖。各查询读取上一层中允许访问的位置；上一层完成后，同一层的已知输入位置可以并行计算。

沿图中的箭头追踪，便能区分两种等待：RNN 在同层等待前一位置，因果 Transformer 在下一层等待所需的上一层结果。已知输入能否并行，取决于这些依赖，而不是“序列”这个名称。生成新的 token 又增加了另一条依赖：下一次调用的输入由本次输出确定。

编码器把已知输入转换为表示，解码器根据允许访问的信息逐步产生输出。现代因果语言模型常用只含解码器的结构 (decoder-only)，本章采用这一结构。处理已知输入时，同层位置可以并行；生成时，第一个输出确定后才能成为下一次输入。因此，输入处理沿网络深度推进，生成还增加了跨调用的串行依赖。¹

练习 2-1 (延伸)：从依赖图判断并行

对于含四个已知输入位置的序列，分别画出三层 RNN 和三层因果 Transformer 的计算依赖图。再增加一个

¹ 序列依赖计算记录。

待生成的输出位置，指出计算该位置之前必须等待哪个节点完成。设每层每位置的计算耗时为一个单位、并行资源充足，分别求完成已知输入的最长依赖链长度；解释增加计算单元可以缩短哪些等待、不能缩短哪些等待。

2.1.2 模型配置与张量结构

先把一个计算节点展开。设本次为 m 个输入 token 计算投影，每个 token 用一个包含 k 个特征的向量表示。把这些向量逐行排成输入矩阵 X ，它就有 m 行、 k 列，输出宽度为 n ：

$$X \in \mathbb{R}^{m \times k}, \quad W \in \mathbb{R}^{k \times n}, \quad Y = XW \in \mathbb{R}^{m \times n}.$$

每个输出元素是长度为 k 的点积，输出共有 mn 个元素，因此

$$N_W = kn, \quad F = 2mkn, \quad M_W = b_W kn.$$

N_W 是权重参数的数量， b_W 是每个参数占用的字节数。行数 m 翻倍，运算量翻倍，权重大小保持不变；输出宽度 n 翻倍，权重数量与运算量都翻倍。这两个变化分别对应“同一模型处理更多输入”与“模型的一层变得更宽”。下面的模型表都可以用这条规则逐行计算。

先只看 Qwen3-8B 的一条数据路径。一次**前向执行**是使用当前权重从输入算到输出的过程；隐藏向量是网络内部传递的特征表示，其维度给出每个位置保存多少个。前馈网络（feed-forward network, FFN）对每个位置分别做特征变换；**注意力**则在位置之间选择并汇总信息。这两类运算交替组成网络的一层。

Qwen3-8B 包含 $L = 36$ 层，隐藏维度为 $d = 4096$ ，FFN 中间维度为 $f = 12288$ ；注意力把特征分成多组并分别建立位置间的关系，每一组称为一个头。这里有 32 个查询头和 8 组供查询使用的键值头（KV 头），每头 128 维；2.1.3 将沿查询、键和值的路径解释它们。词表包含 151936 个 token，词嵌入是一张把 token 编号映射为向量的查找表，输出头则把末层向量变成每个候选 token 的分数。两者各自保存一份权重。各层参数与这两份词表矩阵相加，模型共有约 81.9 亿个参数。²

一次前向执行从 token ID（词元在词表中的整数编号）开始。词嵌入为每个 ID 查出一个 4096 维向量，向量依次经过 36 层注意力与 FFN，最后经过归一化和词表投影产生 logits，即尚未转换为概率的候选 token 分数。归一化按向量的统计量调整各特征的尺度，使逐层计算的数值保持在适当范围内。词嵌入按 ID 选择行，一个 token 取出一条向量，整个词表则作为常驻数据供不同 token 查询。词表输出头则将隐藏向量与词表矩阵相乘；只生成下一个 token 时，通常只需要最后一个输入位置的 logits。

设同时处理 B 条等长请求，每条本次输入 P 个已知 token。投影和 FFN 可以把它们的输入表示为 $X \in \mathbb{R}^{BP \times 4096}$ ，每一行是一个 token 在当前层的表示，共有 $m = BP$ 行。已有上下文中从缓存读取的键和值不计入这里的 m 。每条请求只补入一个 token 时， $m = B$ 。第一种情形容易形成行数较大的矩阵，第二种在小 batch 下接近矩阵与向量的乘法。权重相同，输入形状却明显不同。

下面比较两种输入形状：一条请求一次处理 8192 个输入位置，以及一条已有 8192 个上下文位置的请求再处理一个新位置。用 B 表示请求数， S 表示调用前已有的上下文长度， P 表示本次输入长度， d 表示隐藏维度。前者取 $B = 1, S = 0, P = 8192$ ，后者取 $B = 1, S = 8192, P = 1$ 。

²Qwen3-8B 配置与参数计算，参数总数为 8,190,735,360。

配置项	Qwen3-8B 参数	对资源需求的直接影响
层数 L	36	重复的计算、逐层状态与深度依赖
隐藏维度 d	4096	子层输入输出与投影尺寸
FFN 中间维度 f	12288	三个 FFN 矩阵的权重与计算
Q / KV 头数	32 / 8	查询计算与上下文保存宽度
头维度 d_h	128	每个头的点积与状态宽度
词表大小 V	151936	嵌入和输出头规模

例题 2-1：从矩阵尺寸求一层的参数量与运算量。 Qwen3-8B 的查询和输出投影宽度为 4096，键和值投影宽度为 1024，FFN 中间维度为 12288。求四个注意力投影与三个 FFN 矩阵的参数量，以及为本次输入的 m 个 token 完成这些投影和 FFN 计算所需的运算量。每个 token 在这一层都有一个 4096 维向量，作为输入矩阵的一行； m 统计本次实际参与计算的 token 位置数。同一词元出现在两个位置时计作两个 token。若有 B 条等长请求、每条本次处理 P 个 token，则 $m = BP$ 。例如，一条请求处理 128 个输入 token 时 $m = 128$ ；八条请求各推进一个 token 时 $m = 8$ 。

解：查询与输出各使用一个 4096×4096 矩阵，键和值各使用一个 4096×1024 矩阵，三个 FFN 矩阵各有 4096×12288 个参数。因此，

$$\begin{aligned}
 N_{\text{proj}} &= 2 \times 4096^2 + 2 \times 4096 \times 1024 = 41,943,040, \\
 N_{\text{FFN}} &= 3 \times 4096 \times 12288 = 150,994,944, \\
 F_{\text{linear}} &= 2m(N_{\text{proj}} + N_{\text{FFN}}) = 385,875,968m.
 \end{aligned}$$

注意这里尚未使用上下文长度：这些投影对每个 token 的特征向量做相同的变换。下一节将介绍查询与上下文之间的注意力计算，其运算量与位置对的数量有关。将两类工作分开，就能预测“增加本次参与计算的 token 数”和“让每个 token 读取更长的上下文”为什么有不同的代价。

符号	含义与使用范围
B, P, S	请求数、本次每请求输入位置数、已有上下文长度
$m = BP$	本次参与投影的 token 总数，也就是输入矩阵行数；普通 decode 时 $P = 1$ ，故 $m = B$
d, f	主干隐藏宽度、前馈网络中间宽度
n_Q, n_{KV}, d_h	查询头数、KV 头数、每头维度
N_{pair}	每层跨全部请求的有效因果位置对数，不含头数
t_e	分派给专家 e 的 token 数，每个 token 表示占该专家输入矩阵的一行；无丢弃路由时 $\sum_e t_e = mk_{\text{top}}$
m_{out}	实际进行词表投影的 token 位置数；只用每条请求的末位置预测下一个 token 时为 B ，它不等于本次生成的全部输出长度

计算整模型时，先按表格中每项操作的矩阵尺寸求一次调用的运算量，再乘这一类层的数量。词表头另外按实际输出位置数 m_{out} 计算；生成下一 token 时，每条请求只需最后一个位置的分值。归一化、激活与查表操作各有自己的运算方式，下面沿数据流说明它们的作用。

2.1.3 注意力计算与位置表示

先跟踪一个位置。模型从它的隐藏向量产生三种表示：**查询** Q 用来提出“当前需要什么信息”，**键** K 用来与查询计算匹配分数，**值** V 是被选中后汇总到输出的信息。三者都由同一位置的输入经过不同的权重矩阵产生。线性投影就是这样的矩阵乘法：把输入特征变换到另一组坐标。

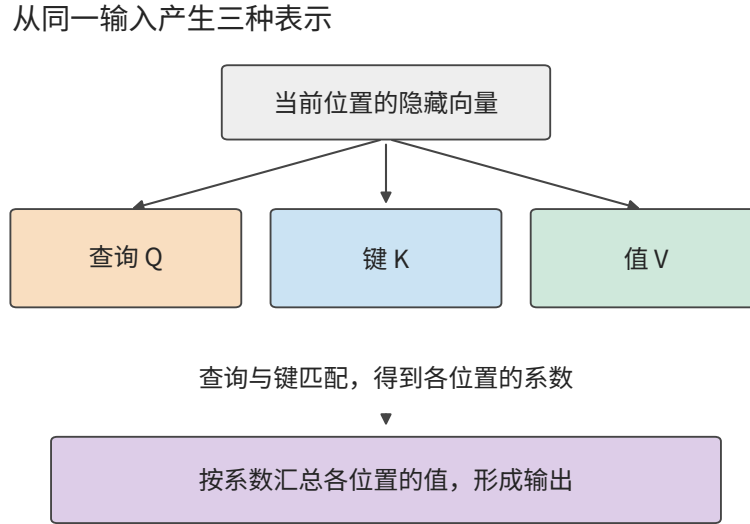


图 2-3 同一位置的输入经不同投影产生查询、键和值。注意力先用查询与键计算位置间关系，再用这些关系汇总值。

采用数学右乘记号，输入 X 分别乘三个权重矩阵，得到 Q 、 K 、 V 。 Q 的总宽度为 $32 \times 128 = 4096$ ， K 和 V 的总宽度各为 $8 \times 128 = 1024$ 。因此， W_q 为 $[4096, 4096]$ ， W_k 和 W_v 各为 $[4096, 1024]$ 。代码中的线性层常以 $(d_{\text{out}}, d_{\text{in}})$ 保存权重；换成数学右乘记号后，输入宽度在前、输出宽度在后，便可逐项对照下面的表格。

上述投影产生了 Q 、 K 、 V ，但尚未让当前位置与上下文交换信息。接下来将 Q 拆成 32 个头，把 K 、 V 拆成 8 个头。每组四个 Q 头共用一个 KV 头。对一个 Q 头，查询向量与允许访问位置的 K 做点积，将点积除以头宽的平方根，使分数尺度随头宽保持稳定，然后屏蔽未来位置。Softmax 把允许位置的分数 s_j 变成正数并归一化，系数为 $a_j = e^{s_j} / \sum_i e^{s_i}$ ；这些系数之和等于一，决定各上下文位置贡献多少信息。这里的“注意力权重”是本次输入算出的系数，区别于训练后保存的模型参数。随后用这些权重对 V 加权求和。各 Q 头的结果拼接后，经 $[4096, 4096]$ 的输出投影 W_o 返回隐藏空间。

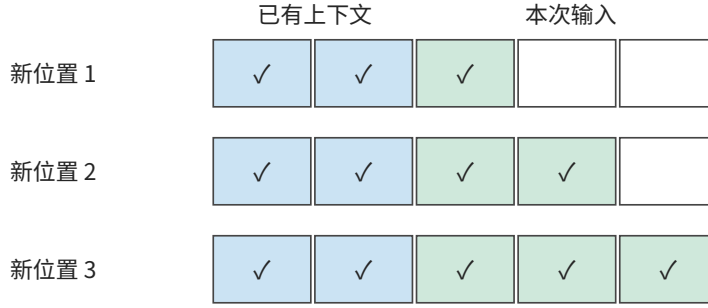
用矩阵记号写出一个头的计算：

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_h}} + \mathcal{M}\right), \quad O = AV.$$

A 是各查询对上下文位置的权重， $\mathcal{M}$ 是因果掩码， d_h 是头维度。点积决定哪些位置与当前查询相关，Softmax 将分数转成加权系数，再用这些系数汇总值向量。Qwen3 在点积前还进行 **查询与键归一化 (QK Norm)** 和 **旋转位置编码 (Rotary Position Embedding, RoPE)**：前者调整各头查询与键的数值尺度，后者通过随位置变化的旋转，使点积包含相对位置信息。

现在数清有多少个点积。输入 P 个新位置、已有 S 个上下文位置时，第 i 个新位置可访问 $S + i$ 个位置，其中 $i = 1, \dots, P$ 。把每个查询允许访问的上下文逐行相加，就得到一个长方形加一个三角形：

已有 2 个位置，再输入 3 个位置



6 个旧上下文位置对 + 6 个新位置对

图 2-4 两段上下文形成长方形加三角形。蓝色为三个新位置分别读取两个旧位置，绿色为新输入内部的因果访问，空白为被屏蔽的未来位置。

$$N_{\text{pair}} = B \left(PS + \frac{P(P+1)}{2} \right)$$

每个位置对上，一个 Q 头的 QK^T 点积约需 $2d_h$ FLOPs，随后 AV 的贡献又约需 $2d_h$ FLOPs。合计所有 Q 头， QK^T 与 AV 的有效矩阵运算量为：

$$F_{\text{attn}} = 4n_Q d_h N_{\text{pair}} = 16384 N_{\text{pair}}$$

在此前没有缓存上下文的 prefill 中，序列长度加倍时，投影的输入行数变为两倍，全注意力的位置对数则约为原来的四倍，因为新加入的位置也要与此前的上下文交互。计算这些位置对有两种常见路径：完整矩形乘法先产生所有分数再屏蔽上三角，因果分块则跳过上三角。分块还可以把评分、归一化和 V 汇总接在一起，让分数在片上被使用后释放。第五章将沿这条数据流解释如何减少中间结果的显存读写。

因此，保持 BP 不变只能保持投影行数不变，不能保持上下文交互不变。例如，把一条长输入拆成两条彼此独立的短输入，原来跨越分界的那些位置对就消失了。相反，仅在同一请求前面增加上下文，会增加每个新位置的点积数，输入投影的行数却保持不变。

2.1.4 前馈网络与前向计算量

注意力负责跨位置的信息交互，FFN 则对每个位置的特征执行非线性变换。SwiGLU 是一种带门控的前馈结构：一条分支变换输入特征，另一条分支为它生成逐元素的调节量。它采用 SiLU 激活函数 $\text{SiLU}(x) = x/(1+e^{-x})$ ，用非线性变换改变特征间的组合方式。Qwen3-8B 使用这种结构：输入分别经过 gate 与 up 两条升维投影，其中 gate 分支应用 SiLU，再与 up 分支逐元素相乘，最后通过 down 投影降回隐藏维度：

$$\text{FFN}(X) = [\text{SiLU}(XW_{\text{gate}}) \odot (XW_{\text{up}})] W_{\text{down}}$$

式中 $\odot$ 表示对应元素相乘；gate 是门控分支，up 是升维分支，down 是降维投影。它们产生的中间向量称为激活。gate 与 up 将每行从 d 维升到 f 维，down 再降回 d 维。三个矩阵因此合计 $3df$ 个参数，处理 m 个

token 的特征向量需 $6mdf$ FLOPs。Qwen3-8B 取 $d = 4096, f = 12288$ ，FFN 约有 1.51 亿个参数，BF16 权重占 288 MiB。升维后较宽的激活也需要临时空间，但只需在相应计算期间保留。

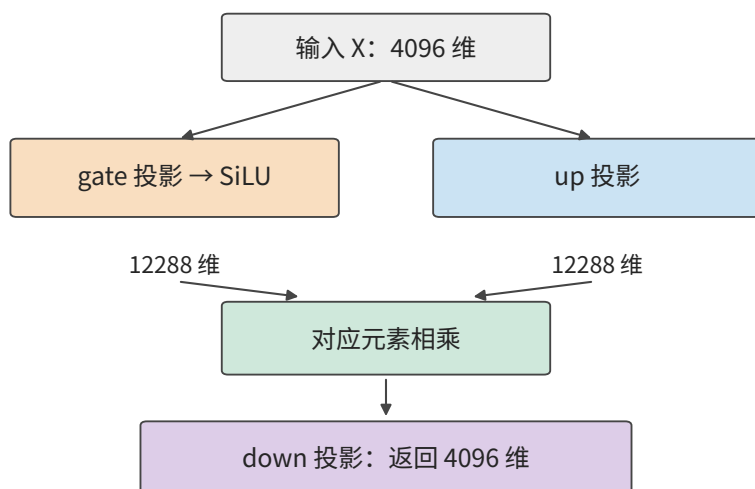
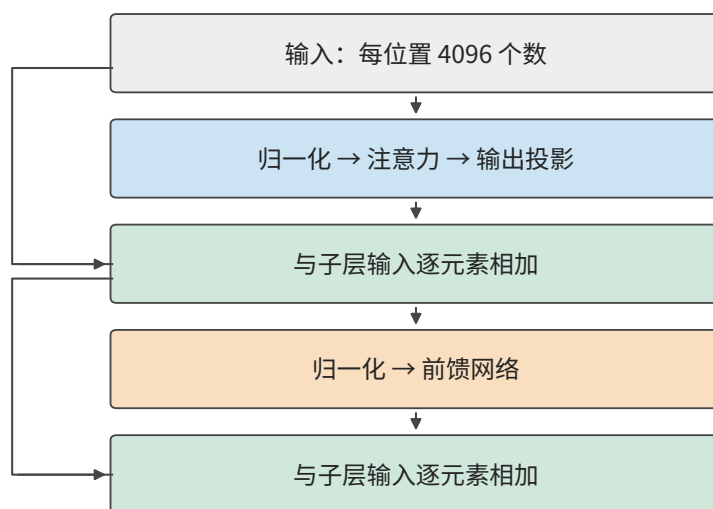


图 2-5 SwiGLU 的两条升维分支。gate 经 SiLU 后调节 up 的对应元素，乘积再由 down 投影降回主干维度。

注意力的 QKV 与输出投影合计约 4194 万个参数。将它与 FFN 的约 1.51 亿相加，FFN 占这些主要投影参数的约 78%，因此短上下文的计算中，三个 FFN 矩阵占据很大份额。随着输入变长，FFN 随位置数线性增长，全注意力则随 $P(P+1)/2$ 增长。两者增长速度不同，因此各部分计算量的占比会随上下文长度变化。

残差连接把一个子层的输入保留下来，在该子层完成变换后，与输出逐元素相加。这样，一条路径执行新变换，另一条路径直接传递已有表示。下面把注意力、前馈和两次残差连接连成完整的一层。

一层先交换信息，再变换特征



左侧旁路保留子层输入，供残差相加

图 2-6 Qwen3-8B 一层的骨架。先完成注意力子层，再完成前馈子层；左侧旁路保留子层输入，与变换结果相加，这种连接称为残差连接。

表 2-1 Qwen3-8B 的单层模块与输出接口

注意力和前馈各行按 36 个主干层累计；词嵌入是词表查行，不再按一次词表矩阵乘法（GEMM，通用矩阵乘法）计算。注意力的 K 、 V 只各有 1024 维，而查询仍有 4096 维。

模型入口

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词嵌入查行	m 个 ID → $m \times 4096$	查表 151936×4096	0；查行访问另计	1

注意力

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
Q 投影：形成查询	$m \times 4096 \rightarrow m \times 4096$	4096×4096	$2 \times m \times 4096 \times 4096$	36
K 、 V 投影：形成可复用上下文	$m \times 4096 \rightarrow m \times 1024$	4096×1024 ，共 2 个	$2 \times 2 \times m \times 4096 \times 1024$	36
QK^T 与 AV ：按内容汇总上下文	32 个查询头，每头 128 维	无新增投影权重	$4 \times 32 \times 128 \times N_{\text{pair}}$	36
输出投影：合并各头	$m \times 4096 \rightarrow m \times 4096$	4096×4096	$2 \times m \times 4096 \times 4096$	36

前馈与连接

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
SwiGLU：逐位置特征变换	$m \times 4096 \rightarrow m \times 12288 \rightarrow m \times 4096$	4096×12288 两个； 12288×4096 一个	$6 \times m \times 4096 \times 12288$	36
归一化、RoPE、激活与残差	保持或逐元素变换上述张量	归一化向量等	按元素计算	36

输出接口

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词表头：转为 token 分数（整模型末端一次）	$m_{\text{out}} \times 4096 \rightarrow m_{\text{out}} \times 151936$	4096×151936	$2 \times m_{\text{out}} \times 4096 \times 151936$	1

表中先看两种增长方式：投影与 FFN 的工作随 m 增长，上下文交互随 N_{pair} 增长。再看 FFN 的三次大投影，便能理解为什么它在短上下文中占有很大的计算份额；上下文变长后，上下文交互的占比逐渐提高。

将每层的线性变换与上下文交互相加，再加入词表头，得到 Qwen3-8B 的主要矩阵运算量：

$$F_{\text{matrix}} = 36 (385,875,968m + 16,384N_{\text{pair}}) + 2m_{\text{out}} \times 4096 \times 151936.$$

式中第一项随输入行数增长，第二项随查询与上下文的位置对数增长，最后一项随实际执行词表投影的位置数增长。对包含 8192 个 token 的输入，按有效因果位置对计算注意力，词表头只处理最后一个位置，共需 133.6 TFLOPs；已有 8192 个上下文位置、再补入一个 token 时，共需 20.0 GFLOPs。³

³逐层记录。

prefill 同时处理 8192 个输入位置，每个位置都要经过投影和 FFN。decode 只对新位置执行这些运算，旧位置的 K 、 V 则从缓存读取。因此，缓存省去了旧位置的投影和 FFN 重算，当前查询与上下文的注意力计算仍要执行。下一节将计算这部分缓存和访问。

练习 2-2 [核心]：输入行数相同，注意力为何不同

使用表 2-1，取 $B = 4, S = 4096, P = 1024$ ，分别计算投影与 FFN、有效因果注意力，以及最后一个位置的词表输出层所需的 FLOPs。保持 $BP = 4096$ ，改为 $B = 1, S = 4096, P = 4096$ ，先预测哪些项相同，再计算差异。最后比较两种输入处理方式：完整处理 8K 输入，或复用前 6144 个位置的 KV、只处理新增的 2048 个位置。求后一种方式节省的矩阵运算比例。

2.2 自回归生成的资源消耗计算

前面已经介绍了一次前向计算包含哪些运算。要计算生成完整回答所需的资源，还需要知道模型调用多少次，以及状态保存多久。同一份权重可能在一个请求中被使用数百次，KV 则一边读取一边增长。最终保存的字节数与累计访问的字节数因此可以相差几个数量级。

2.2.1 Prefill、Decode 与状态复用

模型收到一段已知输入后，可以同时计算多个位置的表示。这一阶段称为 prefill。由最后位置的 logits 选择首个输出 token 后，下一次调用把这个 token 作为新输入，得到再下一个输出；这种逐步扩展的阶段称为 decode。两者使用同一个模型，但前者有多个已知输入位置，后者在普通串行生成中每条请求只增加一个位置。

如果每生成一个 token 都把整个前缀重新送入模型，旧位置的投影与 FFN 会重复执行。因果模型中，旧位置不能看到后来加入的 token；在权重、位置与相关计算条件不变时，旧位置产生的 K 、 V 可以保留。后续查询读取这些上下文 K 、 V ，并补入当前位置的新 K 、 V ，就避免了旧位置的大量重算。

缓存把“重新计算旧位置的表示”变成“保存并访问旧位置的状态”。未来位置会产生自己的 Q ，用它对旧位置的 K 评分，再以分数汇总 V ，所以长期保存的是 K 和 V 。旧查询已经完成了自己的任务，下一步使用新的查询。第八章将进一步讨论如何在不同请求之间共享这些状态。

Qwen3-8B 的 BF16 KV 每个上下文 token 占：

$$\begin{aligned} c_{KV} &= 2Ln_{KV}d_hb_{KV} \\ &= 2 \times 36 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 144 \text{ KiB.} \end{aligned}$$

第一个 2 分别对应键和值； L 层各保存 n_{KV} 组宽度为 d_h 的向量，每元素占 b_{KV} 字节。于是，保存 H 个上下文位置需 $M_{KV} = c_{KV}H$ 。Qwen3-8B 的 $H = 8192$ 对应 1.125 GiB，再处理一个新输入位置就追加 144 KiB。层数或 KV 头数减半，增长速度也减半；batch 增加则为每个独立请求增加一份上下文。⁴

2.2.2 内存占用与数据访问量

KV 缓存通过保存已经算出的结果，减少后续生成中的重复计算。为了分析节省的计算与增加的存储开销，我们按用途和保存时间，把数据分为权重、上下文状态和临时数据三类。

⁴缓存累计计算。

数据类别	保存与复用	本章计算的主要项目
权重	通常在多次调用间保持不变	常驻字节、被访问的权重集合、批内复用
上下文状态	随请求保留，持续追加或更新	实际占用的容量、每步读写量及更新所需的计算
临时数据	在相应算子或阶段中产生，使用后释放	张量尺寸、接口数据量、需要同时保存的部分

Qwen3-8B 全部 BF16 权重约占 16.38 GB，即 15.26 GiB。模型加载后，这份数据供多个请求反复使用。一次前向中，嵌入按 token ID 查行，层内投影与 FFN 使用对应矩阵，末端词表头计算输出分数。访问哪些权重，因而由计算图中的具体操作决定。

对于 $[m, d] \times [d, f]$ ，每一行都使用同一个权重矩阵。将权重块读入片上存储后，可以依次用于多行输入，让一次读取的数据参与更多乘加运算。处理行数 m 增加，计算量随之增加，整份矩阵的容量保持不变。第五章将进一步解释片上容量与分块怎样决定一份权重能够复用多少次。

临时数据占用多少内存，取决于哪些张量同时存在。前一层释放缓冲后，后一层可以复用同一块空间；算子融合还可以让部分中间值只保存在寄存器或片上存储中。因此，计算读写量时要数清每次访问，计算内存峰值时则要找出同时存活的张量。第五章将结合具体程序分析。

存储量取决于某一时刻需要保留哪些数据，访问量则取决于执行过程中读写了多少次数据。例如，一步 decode 若将整段旧上下文读一遍，逻辑读取量为 $c_{KV}H$ ；连续执行多步，就要逐步累计。确定这些访问实际经过哪一级存储后，才能将相应的 R 除以该接口的带宽。

例题 2-2：增加 batch 后，上下文读取量何时超过权重读取量？ Qwen3-8B 每批将所需共享权重读取一遍，数据量约为 15.14 GB，嵌入按请求查行另计。每请求的 8K BF16 上下文为 1.125 GiB，假定各请求独立、旧上下文各读一次。

解：每批共享权重读取量记为 R_W ，各请求的上下文独立，则上下文读取量为 $Bc_{KV}H$ 。两者相等时

$$B_* = \frac{R_W}{c_{KV}H}.$$

以 $R_W = 15,136,811,008$ 字节、 $c_{KV} = 147456$ 字节代入， $H = 8192$ 时最小整数 batch 为 13； $H = 2048$ 时为 51。上下文缩短到四分之一，使上下文读取量超过权重读取量所需的请求数约增至四倍。上下文越长，批内权重复用的收益就越早受到限制，因为每个新增请求都会增加一份独立上下文的读取。⁵

沿第一章的数据路径看，加载把权重从存储送到 GPU 显存，执行再把所需权重送到计算单元。KV 和层间激活也可在设备内部保存与使用。相同的逻辑访问因此可以留在片上、经过 HBM，或跨设备传输；第四章将给这些路径分别配置容量与带宽。

2.2.3 生成完整回答的计算量与状态读写量

上一节比较了一步执行中的读取量。一个完整回答会重复执行许多步，同一份上下文也随之被读取许多次。现在固定一个普通串行生成请求： $B = 1$ ，已恢复前缀为 S ，本次输入 $P \geq 1$ 个 token，最终返回 $G \geq 1$ 个 token。令 $H = S + P$ 。prefill 处理新输入并产生首个输出，随后执行 $n_d = G - 1$ 次 decode。最后返回的 token 尚未重新送入模型，所以请求结束时并不自动为它保存 KV。

⁵ 跨章核对。

每一步追加一个位置，重读已有上下文

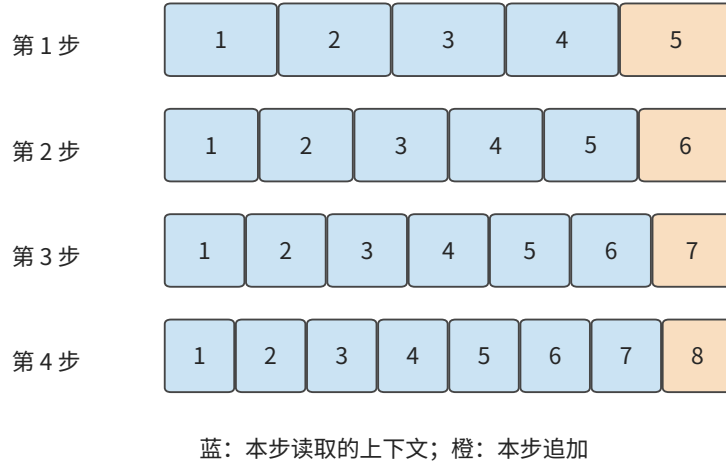


图 2-7 四步生成的上下文读写。每一步读取蓝色的已有位置，再追加一个橙色新位置；本图初始上下文为四，四步后保留八个位置。

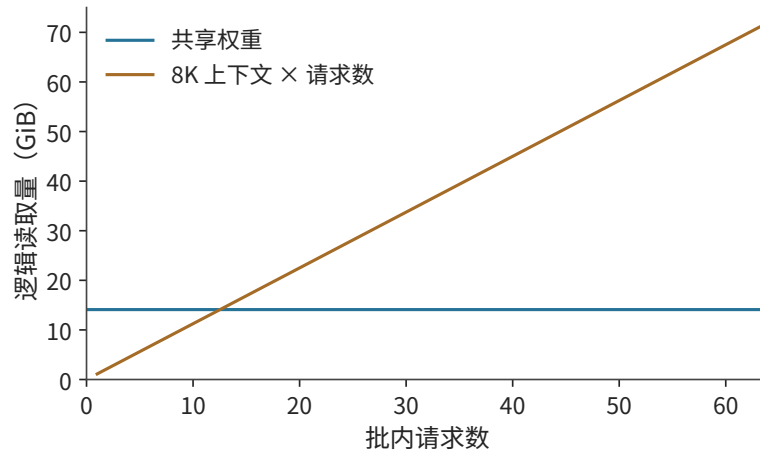


图 2-8 同一批次的主要投影权重可共享读取，各请求则各有一份上下文。图中固定每请求 8K 上下文，分别累计共享权重与独立 KV 的逻辑读取量。

图 2-7 中的每一行都比上一行多一个已填色方块：先前追加的位置现在也成为上下文。第 j 次 decode (j 从 0 开始) 执行前，上下文长度为 $H + j$ 。于是，若每次 decode 将此前的上下文各读取一遍，累计读取量为：

$$R_{\text{old}} = c_{\text{KV}} \sum_{j=0}^{n_d-1} (H + j) = c_{\text{KV}} \left[n_d H + \frac{n_d(n_d - 1)}{2} \right]$$

decode 新增的状态大小为 $c_{\text{KV}} n_d$ ；若保留整个上下文，最终有效 KV 为 $c_{\text{KV}}(H + n_d)$ 。本次请求的全部新增 KV 还应包含 prefill 写入的 $c_{\text{KV}} P$ 。已有前缀的恢复与传输单独核算。求和中的每一项都对应一次 decode 开始时已有的上下文：早出现的位置参与更多后续查询，晚出现的位置参与较少查询。

例题 2-3：逐步生成如何累积出千 GiB 级的 KV 读取量？ 求请求结束时保存的 KV 大小，以及生成期间读取已有 KV 的总量。

解：取 $S = 0$ 、 $P = 8192$ 、 $G = 1025$ ，则 $n_d = 1024$ ，累计访问 8,912,384 个上下文位置。每个位置占 144 KiB，因此累计读取约 1224 GiB。生成期间只新增 144 MiB KV，结束时共保存约 1.27 GiB。累计读取量是最终保存量的近千倍，因为较早生成的上下文被后续查询反复使用。

这组调用也决定了总运算量。线性投影和 FFN 在每次 decode 中处理一行，上下文交互则随 $H + j$ 增长。设每步固定的矩阵运算量为 F_0 、与每个上下文位置交互所需的运算量为 a ，便有

$$F_{\text{decode, total}} = n_d F_0 + a \left[n_d H + \frac{n_d(n_d - 1)}{2} \right].$$

输出变长时，固定工作按 n_d 增长，生成过程中新增的上下文还会使一部分累计运算量按生成步数的平方增长。返回一个 token 时 $G = 1$ 、 $n_d = 0$ ，只有 prefill；这一特殊情形也说明为什么完整回答包含 $G - 1$ 次后续 decode。

图 2-7 沿生成步数累加读取，图 2-8 沿请求数累加读取。多条独立请求的状态容量与上下文访问分别相加，批内权重则共同使用。长度不同时，逐请求代入各自的 H 与 n_d ：长输出让已有上下文被更多次读取，输入越长，第一次 decode 需要访问的上下文就越多。对实际对话，可先根据每轮输入长度、缓存命中情况和返回的 token 数，还原这条调用链，已保存的 Chat 长度核算给出了具体代入过程。

2.3 上下文状态表示与访问机制

这里的上下文状态，是模型处理已有上下文后保存、供后续计算复用的表示，例如 KV 缓存或递推状态。上下文长度按 token 位置计数，状态容量按实际保存的字节数计量。

完整上下文 KV 的容量随序列长度增长，累计读取又随生成次数增加。改变模型结构，可以从几个方面降低这些开销：减少每个上下文位置的表示宽度，压缩或选择上下文条目，或者把上下文信息汇总到固定大小的状态中。这些选择影响的对象不同，也会产生不同的新增工作。比较时需要同时计算状态占用的空间，以及选择、读取和更新状态所需的工作。

2.3.1 多头注意力与 KV 共享

多头注意力让不同的查询头分别学习上下文中的关系。在多头注意力（MHA）中，每个 Q 头对应自己的 K 、 V 头；多查询注意力（MQA）让所有 Q 头共用一组 K 、 V ；分组查询注意力（GQA）则把 Q 头分组，每组共享一组 K 、 V 。共享后，每个上下文位置需要保存的 K 、 V 组数减少，上下文位置的数量保持不变。

先看一个只有四个查询头的单层例子。MHA 为四个查询头各存一份 K 、 V ；GQA 可以让前两个查询头共享第一份 K 、 V ，后两个共享第二份；MQA 则让四个查询头共用一份。若上下文长度和头宽相同，需要保存的 KV 份数依次为四、二、一。因此，GQA 在这个例子中把上下文保存量减半。四个查询头仍可给同一段上下文打出四组不同的分数，这正是“共享上下文表示”与“合并查询”之间的区别。

Qwen3-8B 采用这种共享方式，让每四个查询头共用一组 KV，总计 32 个查询头、8 个 KV 头。如果给每个查询头单独保存 KV，容量会是当前配置的四倍；如果全部查询共享一组，则会当前配置的八分之一。实际 GQA 配置保存 8192 个位置需要约 1.13 GiB。本节末的容量图固定上下文长度与头宽，只改变 KV 组数，因此柱高之比直接呈现共享比例。

共享 K 、 V 改变了被查询的数据，32 个 Q 头仍分别计算分数和加权输出。因此，KV 投影和保存量按组数减少， QK^T 与 AV 交互仍按查询头数累计。一次读入的 K 、 V 还可服务同组多个查询头，从而在实际计算中复用同一份数据。

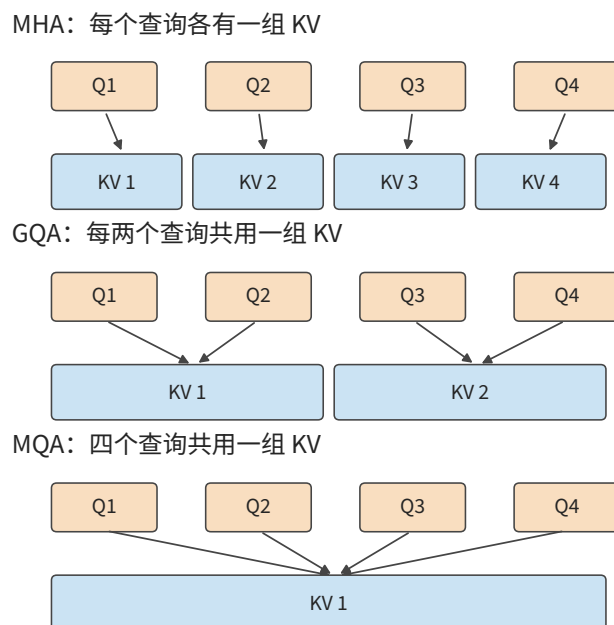


图 2-9 固定四个查询头，分别为每头、每组和全部查询保存 KV。连线表示使用关系；共享上下文后，各查询仍分别产生自己的评分与输出。

可以直接由组数预测资源变化。将 n_{KV} 减半，键值投影参数和上下文状态容量都减半；查询头数 n_Q 不变，查询与上下文交互的运算量仍按原来的头数累计。减少存储与减少计算是两个不同结果，需要分别计算各组件的开销才能判断。

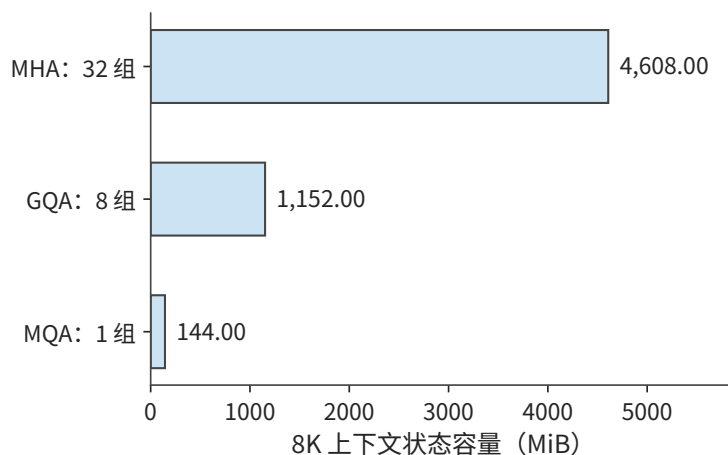


图 2-10 固定 Qwen 层数、头宽、8192 个上下文位置与 BF16，只改变 KV 组数。实际模型采用 GQA；其余两柱是用于分析共享比例的结构变体。

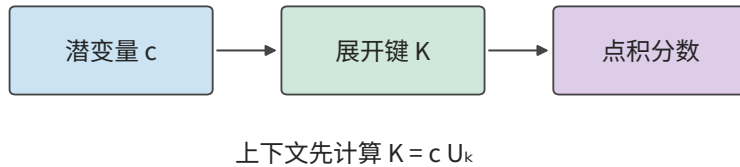
KV 共享也展示了基础设施的限制如何影响模型设计。MQA 的原论文直接把增量推理中的内存带宽开销作为动机；GQA 则在 MHA 的质量与 MQA 的速度之间寻找折中。⁶ 减少 KV 头数既改变模型可表达的注意力关系，也减少服务端必须保存和读取的状态。

⁶MQA 原文摘要；GQA 原文摘要；Llama 3 报告第 6.1 节。容量示例是本书按固定服务状态推导的教学预算，不是这些模型的历史设计记录。

2.3.2 潜变量压缩与 MLA 执行路径

GQA、MQA 通过减少 KV 组数节省空间，多头潜变量注意力（Multi-head Latent Attention, MLA）则通过降低上下文表示的维度节省空间。可以先把输入投影为低维潜变量 c ，再通过上投影得到用于注意力的 K 、 V 。模型在训练中学习这些低维表示与上投影，生成时将上下文以 c 的形式保存。

路径一：先展开每个上下文位置



路径二：先变换当前查询

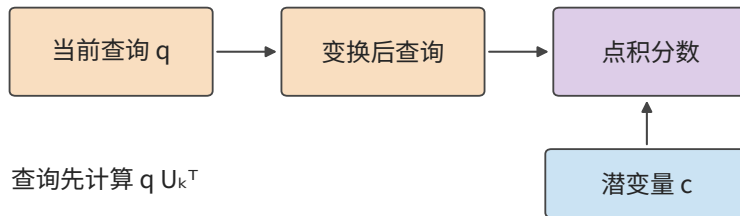


图 2-11 两种计算路径改变上投影的位置。展开路径先恢复上下文键，紧凑路径先变换当前查询，再直接读取潜变量；结合律保证对应点积可以按这两种次序计算。

为什么可以只保存潜变量？设键的上投影为 U_k ，上下文潜变量按行组成矩阵 c ，展开键为 $K = c U_k$ 。矩阵乘法的结合律给出

$$q K^T = q (c U_k)^T = (q U_k^T) c^T.$$

左边先为每个上下文位置恢复键，右边先变换当前查询。对单个新查询，右边只需做一次查询变换，随后直接与潜变量点积。值向量也可在加权汇总后再上投影：先汇总低维向量，再恢复输出维度。这样，原来针对每个上下文位置执行的升维操作，改为针对当前查询和汇总结果执行。

本章采用 Kimi K3 的 **Gated MLA（门控多头潜变量注意力）** 作实例；门控通过学习得到的系数调节注意力输出。紧凑路径保存 512 维潜变量和参与 $Q K^T$ 点积的 64 维额外分支；NoPE 表示该处不使用位置编码的设置，它保持该分支的计算维度不变，省去旋转位置编码。24 个 MLA 层、8192-token 上下文、BF16 下，保存量为 $24 \times 8192 \times (512 + 64) \times 2$ ，即 216 MiB。

展开缓存将潜变量先恢复为各头的 K 、 V ，再保存这些向量；同样的 8192 个位置需要约 11.25 GiB。紧凑缓存则保存上投影之前的潜变量。两者的容量差异因此来自缓存位于线性变换的哪一侧：紧凑缓存每步变换当前查询，展开缓存直接读取各头的上下文向量。⁷

这个变换减小了每个上下文位置的保存宽度，但仍需为每个位置保存一个潜变量向量。因此，存储量和点积运算量仍随上下文长度增长。若上下文很长，紧凑表示节省的读取量更加可观；若上下文很短，查询变换本

⁷Kimi K3 MLA 展开计算。

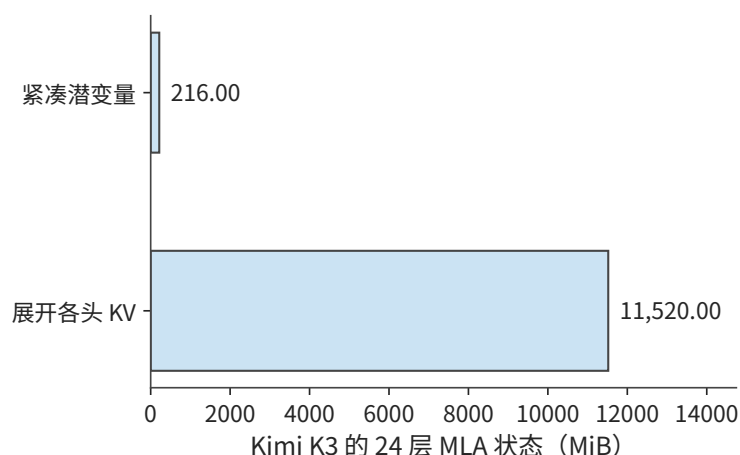


图 2-12 Kimi K3 的 24 层 MLA 在相同 8K 上下文、BF16 下的两种保存量。紧凑路径保存上投影前的表示，展开路径保存各头的键和值。

身的开销占比更高。接下来先讨论如何改变上下文条目的保存和选择方式，再研究把上下文汇总到固定大小矩阵中的方法。

练习 2-3 [延伸]：状态大小与累计读取

取 Qwen3-8B、 $B = 1$ 、 $S = 0$ 、 $P = 8192$ ，分别返回 $G = 513$ 和 $G = 1025$ 个 token。求后续 decode 次数、最终 KV 大小和累计旧上下文读取量。解释输出数量接近翻倍时，新增状态、总状态与累计读取为什么有不同的比例。再将 KV 位宽减半，指出这三项中哪些随之减半。

2.3.3 局部窗口、上下文压缩与稀疏索引

减少上下文表示宽度之后，还可以改变保留哪些上下文、每次访问哪些上下文。局部窗口只关注最近的一段位置；压缩机制把若干位置合成较少的条目；稀疏选择先确定相关条目，再把它们送入主注意力。窗口决定保留的近期范围，压缩决定长期条目的数量，索引决定当前查询读取哪些条目。索引器先扫描用于选择的表示，再把选中的主状态交给注意力，索引扫描和主状态读取都需要单独计算开销。

先把这里的三个对象分开。**局部滑动窗口注意力 (Sliding-Window Attention, SWA)** 直接读取最近的一段位置；窗口长 128 时，较早位置退出这一局部窗口。**全局主 KV** 是供主注意力读取的长期表示，可能已将多个位置合并；“全局”说明它覆盖较长历史，不表示每次都读取全部条目。**索引 K** 是用于检索的另一份较小表示：索引器先用它给条目打分，再让主注意力读取选中的主 KV。索引分数用于选位置，主注意力另用自己的查询计算最终汇总权重。

模型通过训练学会把一段输入汇总为共享表示。一条 512 维主 KV 记录兼作注意力中的键和值，供多个查询头共享，容量按一份记录计算。SWA 保存近期细节，全局分支提供较早信息，两者共同供当前查询使用。

DeepSeek V4-Flash 有 43 个主干层、4096 隐藏维度、64 个注意力头，每头维度 512。 Q 先从 4096 维投影到 1024 维，再得到 64×512 维查询；共享 KV 从 4096 维投影到 512 维；输出采用分 8 组的低秩投影。各投影的输入、输出维度分别确定矩阵形状。

DeepSeek V4-Flash 使用两种压缩注意力：**压缩稀疏注意力 (Compressed Sparse Attention, CSA)** 先压缩上下文，再通过索引选取相关条目；**高压压缩注意力 (Heavily Compressed Attention, HCA)** 把更多位置汇成较少的粗粒度条目，供查询访问。两者都用于减少长上下文的存储与访问量，区别在于压缩程度和是否进一步筛选条目。其主干包含 2 个纯窗口层、21 个 CSA 层和 20 个 HCA 层，窗口长度为 128。CSA 按压缩比 4 保存上下文，并维护索引；HCA 按压缩比 128 保存更粗的上下文。用 BF16 参考表示，CSA 每个完成的压缩条目含

512 维主状态，另有 128 维索引条目。已有 S 个位置时，已生成的条目数按 $\lfloor S/4 \rfloor$ 计算；尚未完成的压缩块还需独立缓冲。

压缩比描述输出条目数，不一定等于一个条目的全部输入范围。V4 的 CSA 每四个新位置生成一条记录，但采用相邻块重叠：除首块的边界处理外，一个条目通过两组不同投影汇总相邻两块、共八个位置的信息；每个位置还使用学习得到的逐通道权重。HCA 每 128 个位置生成一条记录，不采用这种相邻块重叠，也不使用 Top-K 索引，而是读取所有已完成的粗粒度条目。V4.1 的 CSA2 才取消了 V4 CSA 的重叠压缩。

一次 CSA 查询的主注意力最多读取窗口加 512 个压缩条目，但索引打分仍扫描已保存的压缩索引条目。在 $S = 8192$ 时，一个 CSA 层保留 2048 个压缩条目：主注意力状态约 2 MiB、索引约 0.5 MiB，窗口约 0.125 MiB；主注意力选择的压缩条目最多约 0.5 MiB，索引扫描却仍需处理那 2048 个索引条目。HCA 则有 64 个已生成的条目，主注意力状态约 0.0625 MiB，并访问窗口状态与这组粗粒度压缩条目。

将 43 层相加，8192-token 上下文下的窗口、压缩上下文和索引约为 59.125 MiB；另有约 11.641 MiB 的 FP32（每元素 4 字节浮点格式）压缩器缓冲，合计约 70.8 MiB。最后查询的主注意力与索引数据量约 27.6 MiB。状态分解分别列出了这些数据，因此能看出选择条目数、保存量和每步读取量并不相等。

先形成压缩条目，再由查询选择

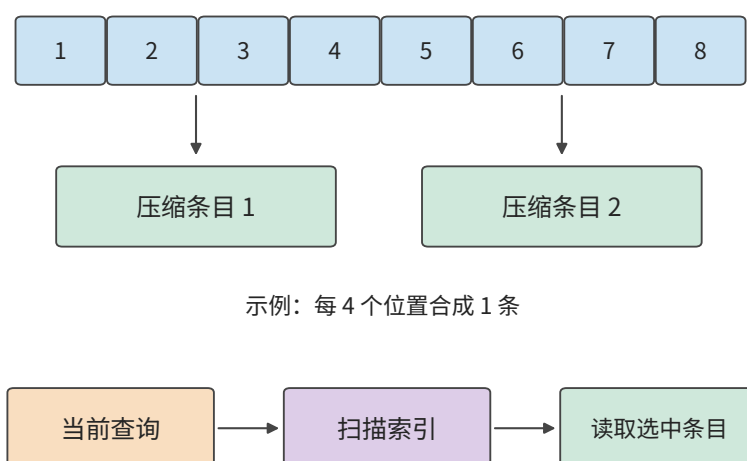


图 2-13 上下文压缩与索引选择的先后关系。上方只示意八个位置产生两条记录的数量关系，省略 V4 CSA 的重叠投影与加权汇总，下方展示一次查询先扫描索引再读取主状态的过程。

压缩还引入一种与查询不同的更新节奏。新 token 到达时，先更新当前压缩块；累计满压缩比要求的位置数后，再生成一个长期条目。CSA 每四个位置完成一块，HCA 每 128 个位置完成一块。因此，在处理同一个输入位置时，主注意力访问多少条目、索引扫描多少条目、压缩器是否完成一块，需要分别计算。第 128 个位置可以同时触发两种压缩块的完成。⁸

练习 2-4 [延伸]：共享、压缩与实际访问

先把 Qwen3-8B 的 KV 头数从 8 改为 4，保持 32 个查询头不变，分别计算 KV 容量、KV 投影参数量，以及查询与上下文之间的运算量如何变化。再对 DeepSeek V4-Flash 的 CSA 层，把上下文从 8192 增至 16384，按压缩比 4、每次最多选择 512 个压缩条目，计算保存条目数、索引扫描条目数和选中条目上限。解释三者为何不按同一比例增长。

⁸缓存后续续算记录。

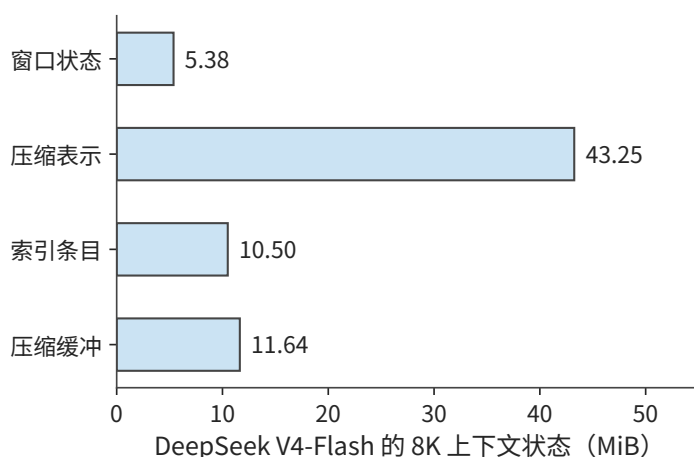


图 2-14 DeepSeek V4-Flash 在 8K 上下文下的四项状态。窗口、压缩上下文和索引保存已处理的上下文信息；压缩缓冲保存尚在汇总或更新中的数据，单独计入容量。

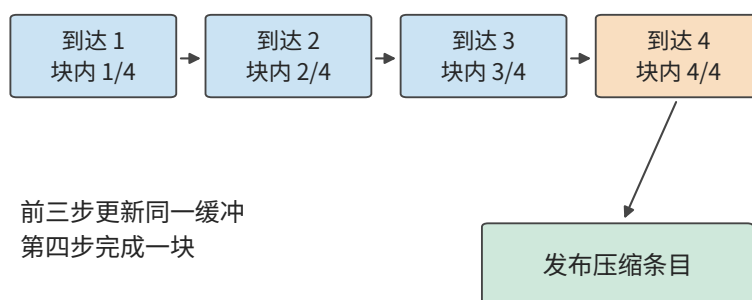


图 2-15 压缩比为四时的一个压缩块的生成过程。前几次输入更新同一块内缓冲，第四个位置到来后形成可供后续查询使用的压缩条目。

2.3.4 线性注意力与有限状态递推

线性注意力通过重新组织查询、键和值的计算，避免显式计算所有位置对的注意力分数。在特征维度固定时，下面这种递推形式处理整段序列的运算量随序列长度线性增长，状态矩阵的大小保持不变。

外积把一个键向量与一个值向量的每对元素相乘，组成二维的关联表。考虑一个由键值外积累加得到的矩阵 S_t 。每个新位置到来时，先读取旧状态，再写入本位置的贡献，当前查询直接从状态矩阵读出结果：

$$S_t = S_{t-1} + k_t v_t^T, \quad o_t = q_t^T S_t.$$

若键宽为 d_k 、值宽为 d_v ，状态始终有 $d_k d_v$ 个元素。位置增加改变的是矩阵内容，而不是矩阵大小。逐位置的上下文列表因此变成了一组累积的键值关联；查询通过这组关联得到输出。

Kimi Delta Attention (KDA) 是 Kimi 使用的一种门控递推注意力，用固定大小的关联矩阵保存上下文信息。它使用门控与 delta 更新；delta 在这里指依据当前预测误差写入修正量。状态更新不仅写入新信息，还根据当前键对既有状态的预测形成修正，并通过衰减控制旧信息的保留程度。直观上，它更新的是一张固定大小

上下文增加，状态矩阵保持同样大小

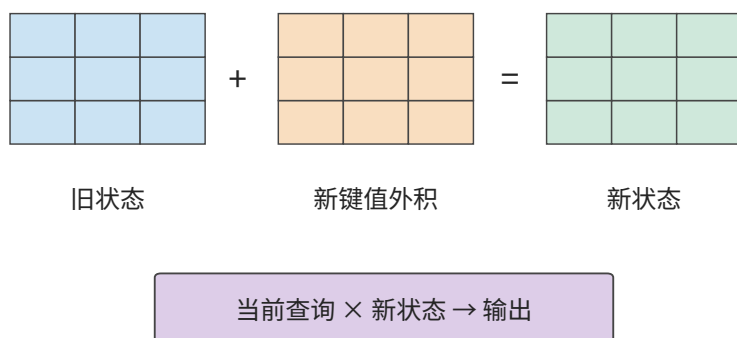


图 2-16 固定矩阵的递推更新。新键和值的外积加入旧状态，矩阵形状保持不变；查询使用更新后的状态计算输出。本图采用简单累加递推。

的关联矩阵，而不是不断向逐 token 保存的列表追加条目。旧信息以汇总后的关联形式保存，当前键控制修正的位置，衰减门决定保留多少旧信息。

Kimi K3 每个 KDA 层有 96 个头，每头递推矩阵为 128×128 。按 FP32 保存，单层为 6 MiB，69 层为 414 MiB。一次 decode 读取旧矩阵、融合新信息并写回，理想情况下，各读写一次的总量就是 828 MiB。这个量不随上下文长度增长：短上下文时它是一笔显著的固定开销，长上下文时则避免了扫描量随位置数持续增加。

prefill 与 decode 的实现方式也不同。decode 使用单步递推；一段已知输入可以通过块式算法并行计算，再在块之间传递状态。块内中间量和块间状态会占用临时空间，因此仅凭最终递推状态的大小，还无法预测 prefill 期间的内存占用峰值。第五章按缓冲寿命分析这一问题。⁹

2.3.5 混合注意力的状态构成

实际模型常把有限状态层与全局注意力层混合使用。这样既利用递推结构的上下文压缩，也保留部分直接访问全局上下文的能力。整个模型的状态由多个部分组成：有的大小固定，有的随上下文长度增长。

Kimi K3 共 93 层，其中 69 个 KDA 层、24 个 MLA 层。8192-token 上下文、紧凑 BF16 MLA、FP32 递推状态及给定 BF16 短卷积槽位下，三项分别为 216 MiB、414 MiB 与 19.4 MiB，合计约 649.4 MiB。若使用参考展开缓存，MLA 一项就变为 11.25 GiB，整体随之改变。¹⁰

Qwen3.6-35B-A3B 同样需要分别计算线性注意力层和完整注意力层的状态占用。其 30 个线性注意力层加 10 个完整注意力层，使每请求每个上下文 token 的全局 KV 为 20 KiB，另有固定 FP32 递推状态 60 MiB，以及保存短卷积所需近期输入的 BF16 缓冲区 1.875 MiB。短卷积先混合邻近位置的信息，缓冲区让下一步更新可以复用这些输入。8192-token 上下文时，三项合计 221.9 MiB。这些状态来自注意力分支；接下来的混合专家 (Mixture of Experts, MoE) 分支准备多个前馈子网络，由路由器为每个 token 选择其中一部分执行，决定同一主干中实际使用哪些权重，两类机制共同组成完整模型。

这几条状态曲线的形状来自不同表示。Qwen3-8B 每增加一个位置，所有 GQA 层都追加 K 、 V ；Kimi K3 只有 MLA 部分追加上下文，KDA 部分保持固定矩阵；DeepSeek V4-Flash 则在压缩块完成时增加条目。对长

⁹块式状态与缓冲记录。

¹⁰混合状态计算。

模型配置决定采用哪种注意力

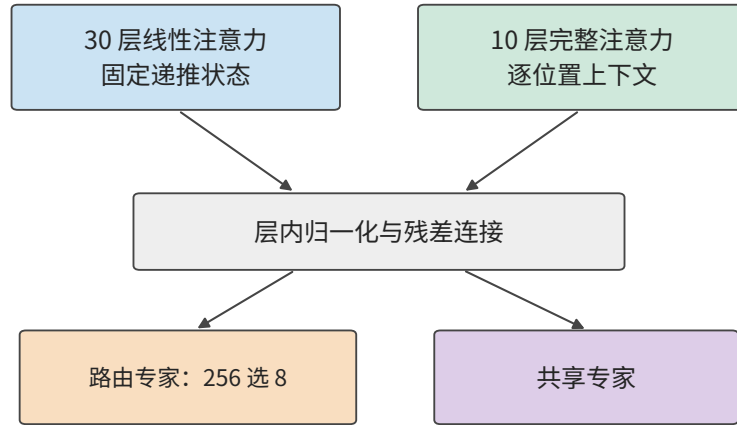


图 2-17 Qwen3.6 的两类层。配置决定每层采用线性或完整注意力，随后都执行路由专家和共享专家；上方两个分支分别表示线性注意力和完整注意力，模型分别有 30 层与 10 层。

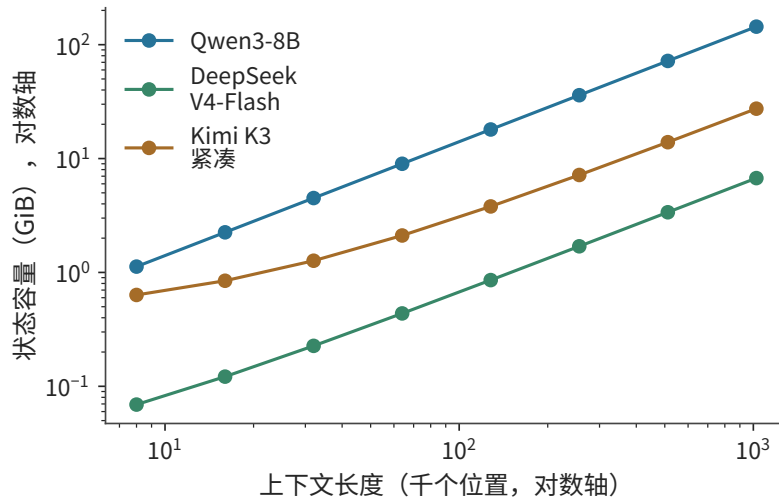


图 2-18 上下文增长时的状态容量。横纵轴均为对数刻度；Qwen 逐位置追加 KV，Kimi K3 紧凑表示包含固定递推状态和增长的 MLA 上下文，DeepSeek V4-Flash 按压缩块增加条目。

上下文，增长速度决定新增容量；对短上下文，递推矩阵等固定部分决定起始占用。

将固定递推状态与逐位置上下文相加，就得到混合结构的容量模型：

$$M_{\text{state}}(H) = M_{\text{fixed}} + cH, \quad H_* = \frac{M_{\text{fixed}}}{c}.$$

H_* 表示两部分占用相等的上下文长度。Qwen3.6 的固定部分为 61.875 MiB，每位置增加 20 KiB，因此 $H_* = 3168$ ；Kimi K3 的紧凑表示每位置增加 27 KiB，固定部分约为 433.4 MiB，交点约在 16.4K。低于交点时，缩短上下文只能节省较小部分；远高于交点时，减少每位置宽度对容量更有效。DeepSeek V4-Flash 则按完成的压缩块增加条目，曲线呈阶梯增长。

这些状态支持不同的信息访问方式：全局 KV 允许查询直接选择旧位置，递推矩阵通过更新保留上下文关

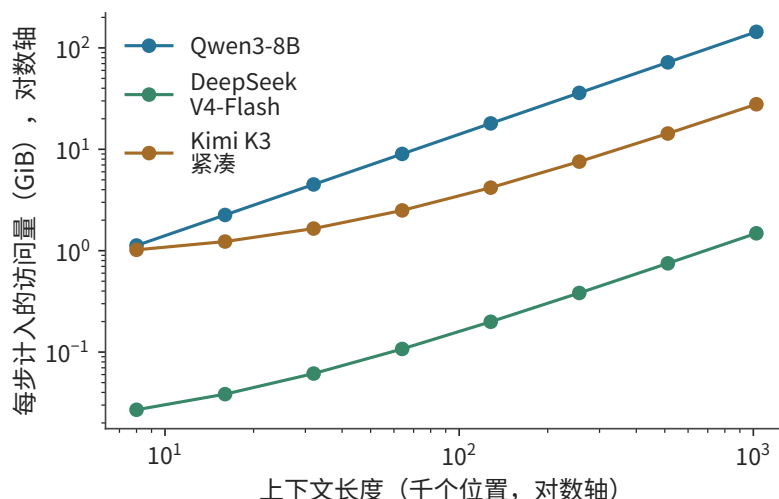


图 2-19 同一组模型每步计入的状态访问。逐位置上下文计读取，递推矩阵计一读一写；这些是指定路径的逻辑数据量，用于比较容量与访问的不同增长方式。

系，压缩索引则在较少条目中寻找相关内容。2.6.3 将结合具体检索任务比较资源需求。¹¹

练习 2-5 [核心]：固定状态与增长状态的交点

对 Qwen3.6，取固定状态 61.875 MiB、每个上下文位置 20 KiB；对 Kimi K3 的紧凑表示，递推状态占 414 MiB，卷积状态占 20,348,928 字节，每个上下文位置另占 27 KiB。分别求固定项与增长项相等的上下文长度，以及 32K、128K 上下文的总状态。若只有 256 MiB 状态预算，两个模型各自最多能保存多少个上下文位置？说明容量预算小于固定状态大小时，为什么缩短上下文仍无法容纳模型状态。

2.3.6 每个 token 存多少，每次 decode 读多少

比较模型的上下文成本，需要同时给出三个量：全局历史每增加一个 token 的平均存储增量、长度为 N 时实际保存的状态，以及一次 decode 查询需要读取的历史。全局 KV 会随上下文增长；滑动窗口只保留最近一段；递推状态的大小固定，却仍要在每一步读写。把三者都除以上下文长度，会掩盖它们不同的增长规律。

下面考虑一条请求，当前查询可访问的上下文共 $N = 8192$ 个位置，包含查询位置自身。表中按各行指定的精度和缓存路径计算全局历史容量，以及一次查询的逻辑读取量：每层选中的 K/V 或共享潜变量读取一次，供各查询头的 QK 与 PV 计算复用，再加上局部 SWA 窗口与索引扫描。递推矩阵、卷积和压缩器的状态更新在表后分别讨论。

¹¹完整模型实验。

模型与缓存路径	全局历史平均增长 (B/token)	8K 全局历史 (MiB)	每次 decode 注意力 KV + 索引读 (MiB)
Qwen3-8B, BF16 GQA	147,456	1,152	1,152
Qwen3-32B, BF16 GQA	262,144	2,048	2,048
Qwen3-30B-A3B, BF16 GQA	98,304	768	768
Qwen3-235B-A22B, BF16 GQA	192,512	1,504	1,504
Qwen3.5-397B-A17B, BF16 全注意力部分	30,720	240	240
Qwen3.6, BF16 全注意力部分	20,480	160	160
Kimi K3, BF16 紧凑路径	27,648	216	216
DeepSeek V4 Flash, 生产合格格式	3,514.25	27.455	12.556
DeepSeek V4.1 Flash, 生产混合格式	890	6.953	11.375

Kimi K3 的两种 MLA 表示展示了缓存路径对容量的影响：表中的紧凑路径每个位置增加 27,648 B，本书采用的 HF 参考实现将 K/V 展开，每个位置增加 1,474,560 B。跨模型缓存计算列出了两种路径的精确字节数。

混合注意力还要加固定状态。在本章 FP32 递推、BF16 短卷积的条件下，Qwen3.5 的递推矩阵与卷积槽合计 184.219 MiB，Qwen3.6 为 61.875 MiB，Kimi K3 为 433.406 MiB。这些固定状态与表中的历史缓存共同构成当前容量；一次 decode 还要读写递推矩阵并更新卷积槽。

理解 V4.1 时，可以先把“生成全局表示”和“查询全局表示”分开。它把 40 层文本主干按顺序分成前 20 层**因果编码器**和后 20 层**解码器**，称为因果编码器—解码器（Causal Encoder-Decoder，CED）。两部分属于同一个自回归模型，编码器中的每个位置只访问当前位置及其之前的信息。

编码器先处理输入。解码器需要的全局 KV 由编码器末层表示经过投影生成，而不必先运行每个输入位置的全部解码器层。解码器每层的局部 SWA 则仍依赖该层自己的输入表示，因此要把提示末尾最多 128 个位置的编码器输出送入解码器，构建局部状态。开始逐 token 生成后，新位置依次经过编码器和解码器的全部 40 层。第三章再计算省下了哪些输入工作，第八章区分编码器和解码器的两种重放。

长上下文的成本还取决于同一段信息在多少层中重复保存。V4.1 的技术报告直接把 HBM、SSD 与主机内存容量，以及 KV 迁移带宽列为设计约束。它没有继续提高 V4 的序列压缩比：V4 使用 4:1 的 CSA 和 128:1 的 HCA，V4.1 取消 HCA，编码器的全局条目按 2:1 压缩，解码器按 1:1 保留。更小的缓存主要来自跨层共享与更低精度，同时保留了更细的历史位置粒度。¹²

共享也改变了各层能够独立选择的信息。复用层不再生成自己的全局 K/V，但每层仍保留独立的查询 Q 和局部 SWA，因而可以对共享条目计算不同的注意力权重，形成新的输出。缓存共享减少了一类逐层独立表示，却没有把所有层的计算变成相同操作。

把上述共享关系代入容量计算。2026 年 9 月发布的 DeepSeek V4.1 Flash 有 40 层文本主干，但只有 4 层拥有独立的全局 KV：从零编号的第 2、8、14 层每两个 token 生成一条压缩记录，第 20 层每个 token 生成一条

¹²DeepSeek V4.1 官方技术报告，第 1、2、3 节与第 6 节；跨章会话的固定条件与复算。

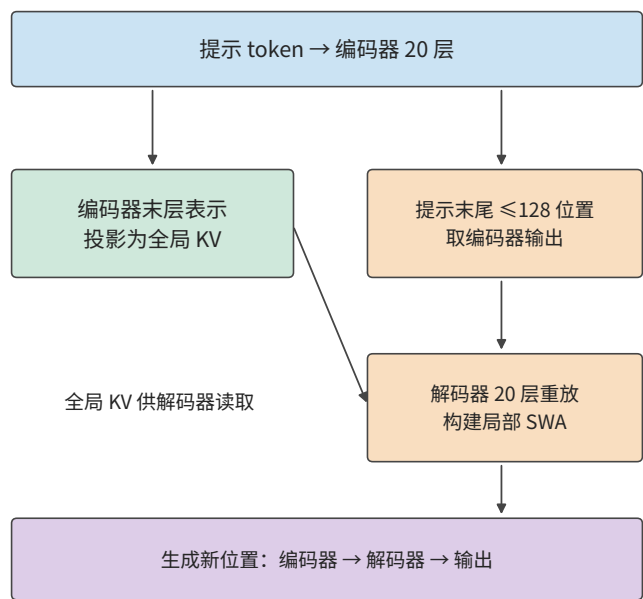


图 2-20 CED 把全局 KV 的生成与解码器局部状态的构建分开。大部分提示位置只执行编码器主体及全局 KV 投影；末尾窗口的编码器输出还经过解码器，近似构建局部 SWA。生成的新位置仍通过全部 40 层。

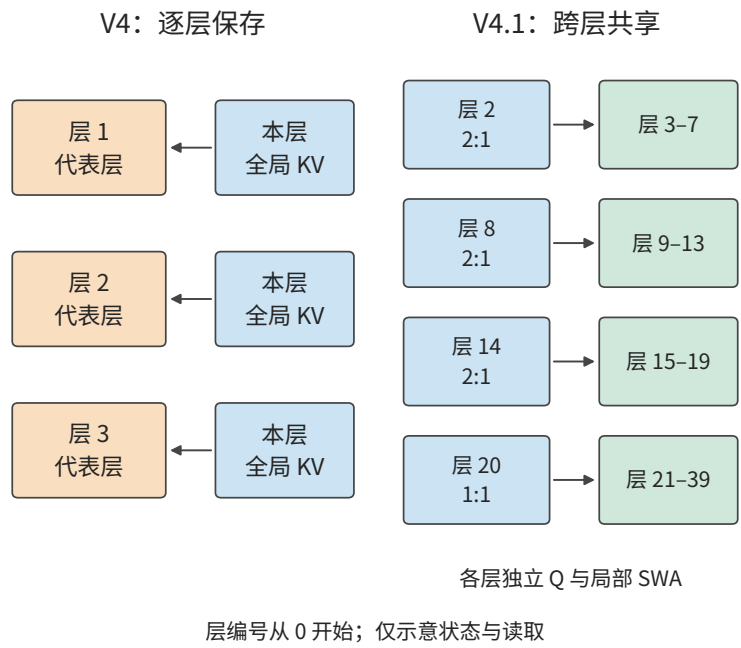


图 2-21 V4 与 V4.1 的全局 KV 保存方式。左侧以三个代表层示意逐层保存；右侧显示 V4.1 四个独立保存全局 KV 的层及共享关系。实线表示数据读取；各使用层仍有自己的 Q 与局部 SWA，图中省略其他计算。

记录，其余全局注意力层共享这些缓存。主记录的 512 个值采用 FP4，每 16 个值另有一个字节的 scale，因此每条记录占 $512/2 + 512/16 = 288$ 字节；索引记录占 $128/2 + 128/32 = 68$ 字节。全局 KV 的平均存储增量为

$$\left(\frac{3}{2} + 1\right) (288 + 68) = 890 \text{ B/token.}$$

旧 V4 Flash 有 21 层 4:1 压缩和 20 层 128:1 压缩。按照官方 FlashMLA 的主记录 584 B、索引记录 68 B 格式，其全局 KV 的平均存储增量为

$$21 \frac{584 + 68}{4} + 20 \frac{584}{128} = 3514.25 \text{ B/token.}$$

两者的全局 KV 平均增量相差约 3.95 倍，对应官方图中取整后的 3,514 与 890 B/token。全局历史之外，V4.1 的局部窗口每层最多保存 128 条记录，每条 FP8+scale 记录占 528 B，40 层合计 2.578 MiB。全局历史随上下文增长，局部窗口则在填满后保持固定容量。

沿生成过程逐步观察，还能看出全局缓存的增长节奏。对 V4.1，全局缓存从偶数长度增长到下一个奇数长度只增加 356 B；到下一个偶数长度时，三个 2:1 压缩器同时完成一条记录，增加 1,424 B。窗口填满后，后续写入会覆盖旧槽位，容量不再增长，但仍然产生写入：40 层 SWA 每步合计写入 $40 \times 528 = 21,120$ B，压缩器更新另计。

共享同一份全局缓存的各层仍需读取数据、部分层还会重新索引；8K 场景下，两代 Flash 的注意力 KV（含局部 SWA）与索引逻辑读取量约为 12.56 和 11.38 MiB，跨层共享主要减少了重复保存的空间。

缓存复用与位置选择还可以分别安排。**压缩稀疏注意力第二版（Compressed Sparse Attention 2, CSA2）**把层分成三种模式。**Full** 层完整执行“生成全局 KV、索引、选取和注意力”的流程。**Reindex** 层共享 KV，用自己的索引查询重新选取 Top-K；**Reuse** 层连同所选位置一起复用。V4.1 的 38 个全局注意力层中，有 4 个 Full、4 个 Reindex 和 30 个 Reuse；另外两个层仅使用 SWA。解码器的首个 Full 层扫描全局，并选出最多 $2048 \times 8 = 16,384$ 个候选位置；技术报告第 2.3.2 节将这个集合称为**候选池（candidate pool）**，其中的候选是可供注意力检索的位置。后续 Reindex 层只在池内各自选取最多 512 个条目。¹³后续层的搜索量因此有了上限，初始全局扫描仍随上下文增长。

例如，某次查询面对 131,072 个全局位置，将它们按每块 8 个分成 16,384 块。首个解码器 Full 层先为所有位置评分，以块内最高分作为块分数，再选出 2,048 块，形成 16,384 个候选位置。它自己仍从全局分数选取 Top-512；后续 Reindex 层在候选内用新的索引查询重算分数，各自选出 512 个位置。候选池限定后续层的搜索范围，各层的 Top-512 决定本层最终读取哪些条目。

候选池让后续层的搜索量有了上限，也使首轮筛选影响后续选择：某个位置一旦落在候选池之外，本次查询的后续 Reindex 层便无法再选入它。模型训练需要适应这一搜索范围，第十章将说明 V4.1 怎样把候选限制纳入后训练。

2.4 条件计算与专家权重复用

前面几节通过共享、压缩和递推减少上下文状态。对另一个主要数据来源——模型权重——也可以问同样的问题：每个 token 是否必须使用模型中的全部参数？下面从 FFN 出发分析这一问题。

稠密 FFN 对每个 token 使用同一组矩阵；MoE 把参数分配给多个专家，由路由器为每个 token 选择少数专家。这样，“模型保存多少参数”与“一个 token 激活多少参数”就可以分别调整，也让 batch 内不同 token 的选择影响实际权重访问集合。

¹³DeepSeek V4.1 技术报告，第 2.3.2 节 Hierarchical Sparse Indexer 使用 candidate pool 和 candidate positions；Figure 5 展示共享候选池与后续 Top-K 的区别。

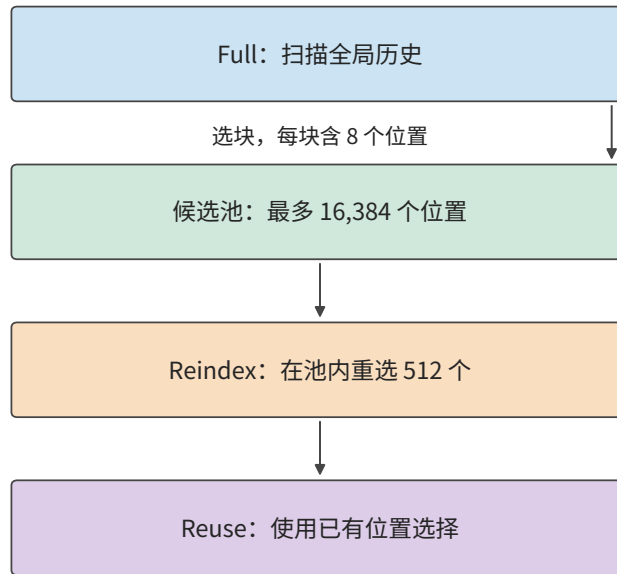


图 2-22 V4.1 解码器的层级位置选择。首个 Full 层先扫描全局，构造最多 16,384 个候选；后续 Reindex 在候选内重新选 512 个，Reuse 使用已有选择。示意位置数量不按比例；各层独立 Q 与局部 SWA 未画出。

2.4.1 稠密前馈网络与专家路由

一个典型 MoE 分支先从输入计算路由分数，选择路由分数最高的 k 个专家 (top- k)，再把 token 送入选中专家，最后按路由权重合并输出。有共享专家时，还会有一条所有 token 都参与的分支。专家本身通常仍由 gate、up、down 三个矩阵构成，因此可以按 2.1.4 的 SwiGLU 计算逐个展开。

Qwen3.6-35B-A3B 的文本主干包含 40 层，隐藏维度 2048；每层 256 个路由专家，每 token 选 8 个，专家中间维度为 512，另有共享专家。一个路由专家包含 $3 \times 2048 \times 512 = 3,145,728$ 个参数，以 BF16 保存为 6 MiB。每层全部路由专家占 1.5 GiB，一个 token 选中的八个专家占 48 MiB。

设专家 e 实际收到 t_e 个 token，其 gate/up 是 $[t_e, 2048] \times [2048, 512]$ ，down 是 $[t_e, 512] \times [512, 2048]$ ，主要矩阵运算量为 $6t_e \times 2048 \times 512$ 。将各专家的运算量相加，就得到全部路由专家的矩阵运算量。路由器先决定各个 t_e ，选中专家执行三次投影，随后收集并加权合并结果；共享分支对全部输入行执行。

用 E 表示路由专家总数、 k_{top} 表示每 token 选中的专家数，单个专家有 N_e 个参数，则每层保存 EN_e 个路由专家参数，单 token 执行的专家矩阵工作约为 $2k_{\text{top}}N_e$ 。增大 E 扩充了可选择的参数集合；只有实际选中数或专家尺寸增加，才会直接增加这部分单 token 运算。Qwen3.6 的 A3B 名称体现激活规模，全部文本参数约为 346.6 亿，BF16 权重约为 69.321 GB。

2.4.2 实际运算量与批内权重读取量

例题 2-4：相同专家计算量，权重读取为何相差 32 倍？ 对 Qwen3.6 的同一层，比较均匀分派和集中分派。

解：即使激活参数相同，batch 内的权重读取也可能不同。假设同时为 $B = 64$ 条请求各执行一步 decode，每条请求本次输入一个 token，因此 $m = B = 64$ 。每个 token 选择 8 个专家，总共有 512 次 token-expert 分派。若均匀分到 256 个专家，每个专家平均接收 2 个 token，当前批次会访问全部路由专家；若所有 token 都集中到同 8 个专家，则每个专家接收 64 个 token，当前批次只访问 8 份专家权重。

设本批实际访问的不同专家数为 U 。未丢弃任何分派的路由满足 $\sum_e t_e = Bk_{\text{top}}$ ，因此

$$F_{\text{expert}} = 2N_e \sum_e t_e = 2N_e B k_{\text{top}}, \quad R_{\text{expert}} = b_W N_e U.$$

第一式按分派的总行数累计，第二式按被读入的不同权重累计。例中两种路由方式都产生 512 次 token 到专家的分派；一个 token 发给八个专家，会在八个专家的输入矩阵中各占一行。这 512 行来自 64 个原始 token，矩阵运算量相同；但 U 分别为 256 和 8，每层读取量分别为 1.5 GiB 和 48 MiB，相差 32 倍。40 层所有路由专家合计保存 60 GiB，集中路由每批选中的权重合计只有 1.875 GiB。

$B = 64$ 的路由情形	每层访问的不同专家	每专家 token 数	每层路由专家理想读取
均匀分配	256	2	1.5 GiB
集中于同 8 个专家	8	64	48 MiB

分派次数相同，访问到的专家可以不同

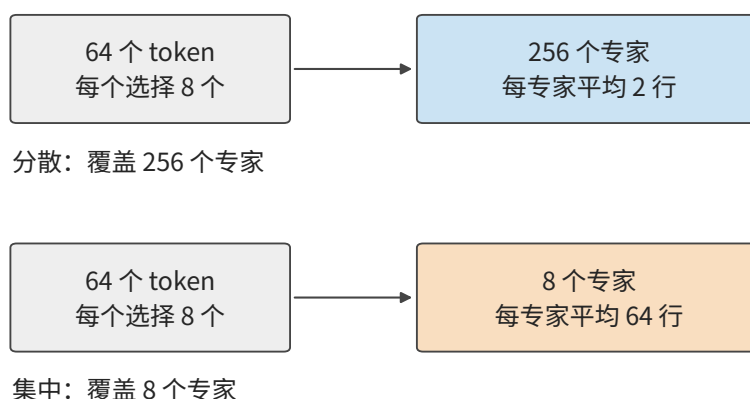


图 2-23 64 个 token 各选八个专家，总计 512 次分派。分散时可覆盖 256 个专家，集中时只访问八个；每专家处理的行数随之改变。

读取差额来自每份权重服务的输入行数：均匀分派时每个专家处理两个 token，集中分派时每个专家处理 64 个 token。同一份权重支持的计算增加 32 倍，理想权重访问量便降至原来的三十二分之一。与此同时，原来要执行许多输入为两行的矩阵乘法，现在只需执行少数输入为 64 行的矩阵乘法；权重复用和计算效率因此都会受到影响。¹⁴

Qwen3.6 在已有 8192-token 上下文的 $B = 1$ decode 中约需 7.33 GFLOPs，小于 Qwen3-8B 的约 20.0 GFLOPs。整个模型的计算量差异来自几项结构上的变化：2048 维主干缩小投影，30 个线性层使用递推状态，MoE 每次只执行选中的专家及共享分支。专家数量增加使总参数量增大，而每次调用的工作量取决于实际执行了哪些运算。

稀疏激活把总参数容量与每 token 的激活计算分开，使模型可以用较少计算访问较大的参数集合。服务时，路由结果决定每份专家权重供多少 token 复用：少量 token 分散到很多专家，会产生许多小矩阵，每次读入的权重也很少得到复用；集中到同一组专家，则能组成更大的矩阵。例题 2-4 展示的正是这两种访问方式的差别。

¹⁴均匀路由；集中路由。

2.4.3 共享专家与潜空间计算

上述差距由专家接收的输入行数决定。接下来还要考虑专家本身的尺寸：共享专家处理全部输入，潜空间专家则先缩小输入维度。两者都会改变一个专家需要多少权重、执行多少运算。

DeepSeek V4-Flash 每层有 256 个路由专家，每 token 选 6 个，另有 1 个共享专家；路由专家中间维度为 2048。单专家的 gate/up 为 $[t_e, 4096] \times [4096, 2048]$ ，down 为反向尺寸，参数为 25,165,824。前三层采用哈希路由，后续采用评分路由：哈希路由按 token 确定专家，评分路由根据当前表示选择专家。

共享专家处理本次全部 m 个 token 的特征向量，路由专家 e 只处理分派给它的 t_e 个 token。若共享专家有 N_s 个参数，一层的专家矩阵运算量为 $2mN_s + 2 \sum_e t_e N_e$ 。普通单步 decode 中，每条请求输入一个新 token，此时 $m = B$ ；prefill 则要累计各请求本次处理的全部 token。DeepSeek V4-Flash 每 token 选择六个路由专家并执行一个共享专家，正是这两个部分相加；容量则需要保存全部 256 个路由专家和共享分支。

Kimi K3 每个 MoE 层有 896 个路由专家，每个 token 选择其中 16 个。输入先从 7168 维主干空间投影到 3584 维潜空间，再经过中间维度为 3072 的专家变换，合并后投影回主干空间。共享专家直接处理主干表示。降维减少了路由专家的矩阵尺寸，同时增加了进入和离开潜空间的两次投影。¹⁵

如果专家分布在不同设备上，就要把输入 token 发送到对应设备，专家完成计算后再合并结果。通信量因此取决于专家放在哪里、输入保存在何处。第六章讨论多卡划分，第九章介绍 CPU / GPU 放置，以及将注意力（attention）与前馈网络（FFN）交给不同资源的 AF 分离。本节先确定每个专家处理的 token 数、矩阵尺寸和权重读取量。

练习 2-6 [延伸]：专家总量、激活与复用

对 Qwen3.6 的一层，取 $B = 64$ 、256 个路由专家、每 token 选 8 个。分别考虑均匀分派和集中分派到同一组 8 个专家的情况，写出各专家接收的 token 数 t_e ，再计算 FLOPs、被访问的专家数量与 BF16 权重读取量。再将专家数减半、专家中间宽度加倍、每 token 选中数减半，判断总专家参数与激活矩阵运算是否变化。最后说明共享专家与潜空间进出投影应加在哪一项。

2.5 网络形状与计算组织

上下文状态与专家选择改变了主要数据集合，网络的深度、宽度和连接方式则进一步决定这些工作使用多大的矩阵、按什么顺序执行。参数量相近的模型也可以有不同的计算图，甚至相近的矩阵 FLOPs 也可能伴随不同的缓冲寿命和串行步骤。

2.5.1 深度、宽度与专家粒度

先忽略词表和小向量参数。若 FFN 宽度与隐藏维度保持固定比例，稠密主干的参数数量大致正比于 Ld^2 。把隐藏维度减半、层数增至四倍，可以保持这部分参数量相近，却把逐层依赖加长了四倍。每个矩阵更小，但必须经过更多层；每层还要保存相应的上下文状态。层数与宽度由此在相近参数预算下改变了计算粒度和依赖深度。

专家结构也可以作类似调整，同时保持参数量和运算量不变。单层路由专家的总参数为 $3Edf$ ，单 token 专家运算量为 $6k_{\text{top}}df$ 。将专家数 E 减半、中间维度 f 加倍、选中数 k_{top} 减半，这两个量都不变。Qwen3-235B-A22B 的 128 个宽度 1536 的专家、每 token 选 8 个，可以与 64 个宽度 3072 的专家、每 token 选 4 个作这一对照。路由输出维度与每专家接收行数仍会改变，从而改变分组矩阵乘法的形状。

在 16-token 算例中，三种专家粒度的专家矩阵运算量保持相同，基线与 64 专家变体均约为 454 GFLOPs。专家数减半、单专家宽度加倍时，相同数量的权重被分成更少的大矩阵；专家数加倍、宽度减半时，则成为更多

¹⁵Kimi K3 专家计算。

的小矩阵。路由器要为每个专家计算路由分数，因此其参数也随专家数变化。专家粒度计算将路由与专家矩阵分别累计，呈现同一专家参数总量如何形成不同的执行形状。

形状也决定多卡分工的最小单位。128 个专家可分成八组，每组 16 个；四个 KV 头按完整头分工时只能形成四份，八卡执行就需要复制部分 KV 或调整分组。第五章从矩阵分块分析执行效率，第六章将进一步把这些矩阵和状态分配到设备。

2.5.2 残差连接与跨层状态

除了层数和宽度，还要看各层怎样连接。残差连接把子层输入直接加到变换结果上，使原始信息和梯度具有跨层直通路。普通残差计算 $x_{\ell+1} = x_{\ell} + f_{\ell}(x_{\ell})$ ，其中 f_{ℓ} 是第 ℓ 个子层的变换，因此子层计算期间要保留输入 x_{ℓ} ，等变换结果产生后才能相加。若后续多个层还会引用同一表示，它的寿命就要延长到最后一次使用。连接方式由此直接决定哪些张量需要同时保存。

普通残差把不同深度的信息不断相加到同一条通路中。为了让层间信息有更多组合方式，同时保持深层网络中的信号传播稳定，DeepSeek V4-Flash 使用 **mHC (Manifold-Constrained Hyper-Connections, 流形约束超连接)**。它把一条残差通路扩展成多条，并约束通路之间的混合系数；这里的“流形约束”具体表现为将残差混合矩阵的元素限制为非负、各行与各列的和接近 1。¹⁶

该模型保存 4 路残差状态，每一路都是当前 token 的一份 4096 维中间表示。保留四路表示，是为了让后续层学习如何组合不同通路的信息。每个子层先将四路表示混合为 4096 维子层输入，执行注意力或 FFN，再把结果写回残差通路。因此，主体矩阵保持 4096 维输入，新增工作集中在进入和离开子层的混合与汇合。四路状态扩大了跨层表示，混合操作负责连接它们与原有子层。

处理 8192 个输入 token 时，mHC 的混合投影约需 555 GFLOPs，普通算术约需 149 GFLOPs。残差混合矩阵先经 **Sinkhorn 迭代**归一化，即交替将每行、每列除以各自的和，使行和与列和逐渐接近 1；这使多层混合中的数值尺度更稳定。归一化后的系数再用于组合各条残差通路；末端汇合这些通路，形成送入词表头的表示。因此，多路残差不仅增加要保存的表示，还增加每层的混合与归一化计算。¹⁷

Kimi K3 希望当前层能够按需要选取不同深度的信息，而不只是接收逐层相加后的一个混合结果。它使用 **Attention Residuals (AttnRes, 注意力残差连接)**，对较浅层的表示计算权重，再加权求和，形成当前层的输入。这种注意力沿网络深度选择信息，前面的序列注意力则在不同 token 位置之间选择信息。为减少保存和读取所有层输出的开销，块式实现把若干层组织成一块，保留供后续块选择的表示。¹⁸

后面的层还要使用这些块表示，所以它们需要在本次前向中继续保存；前向结束后即可释放。上下文 KV 则要保留到后续生成步骤。两者的寿命不同：AttnRes 增加一次前向中的临时存储，KV 随请求跨多次调用保留。¹⁹

2.5.3 多 token 预测与辅助计算

普通自回归主干根据当前上下文预测下一个 token。**多 token 预测 (Multi-Token Prediction, MTP)** 在此基础上增加后续多个位置的预测目标或辅助预测模块。训练时，这些额外目标促使当前表示包含对更远后续内

¹⁶DeepSeek V4 技术报告第 2.2 节说明 mHC 的多路表示与稳定传播，第 2.3 节说明 CSA / HCA；Kimi K3 技术报告说明 Attention Residuals 沿深度选择信息的动机与块式实现。

¹⁷mHC 运算量。

¹⁸DeepSeek V4 技术报告第 2.2 节说明 mHC 的多路表示与稳定传播，第 2.3 节说明 CSA / HCA；Kimi K3 技术报告说明 Attention Residuals 沿深度选择信息的动机与块式实现。

¹⁹逐层调度记录。

普通残差：保留一条旁路



一条原输入与子层输出相加

mHC：保留四路，再混合返回



四路先汇合供子层使用，随后混合子层输出

图 2-24 普通残差与四路 mHC 的连接范围。普通残差保留一条子层输入旁路；mHC 汇合多路状态供子层计算，再混合输出。两种图中的子层输入均为 4096 维。

容有用的信息；推理时，辅助模块可以先提出多个候选 token，再由主模型验证，以减少生成同样数量 token 所需的串行轮数。

推测解码 (speculative decoding) 先用较低开销生成候选序列，再让目标模型验证这些候选。用于推测解码时，MTP 根据当前主干特征预先生成后续 token，再交给目标模型验证。如果多个 token 通过验证，一轮就能产生多个有效输出；否则从最后一个通过验证的位置继续生成。因此，加速效果取决于草稿生成、目标模型验证各需多久，以及每轮有多少 token 被接受。第八章将计算这三者的关系。

DeepSeek V4-Flash 的 MTP 辅助块接收主干特征和 token ID，预测后续位置。对单个 token，即 $B = 1, T = 1$ ，一次辅助计算约需 1.80 GFLOPs，其中共享词表头约需 1.06 GFLOPs。共享词表头占这次计算的一半以上：虽然它不增加一份独立权重，每次调用仍要执行词表投影。²⁰

辅助预测提醒我们，一次前向执行、一次采样和一个有效输出可能是不同的单位。第三章将把编码、生成、迭代及实时输出组织成完整负载；第八章再计算生成草稿与验证所需的时间，以及由此节省的生成时间。

设一次普通 decode 耗时为 T_d ，一轮推测解码的草稿和验证合计耗时为 $T_s + T_v$ ，平均得到 a 个有效输出。每输出分摊时间为 $(T_s + T_v)/a$ ，有收益的条件是

$$a > \frac{T_s + T_v}{T_d}.$$

例如，普通 decode 为 20 ms，草稿与验证合计为 30 ms，平均每轮得到两个有效输出时，每输出分摊 15 ms；若只能得到一个，则需要 30 ms。多预测几个位置增加了单轮工作，只有足够多的输出被采用，才能抵偿这些工作。

²⁰DeepSeek V4-Flash MTP 计算。

2.6 整个模型的资源需求与比较

前面介绍了三类结构变化：上下文表示决定状态大小和访问方式，专家选择决定每次使用哪些权重，网络深度与连接方式决定计算顺序。下面将这些部分合起来，比较完整模型在相同输入条件下的计算量与存储需求，再用具体任务检查模型输出。

2.6.1 各组件的资源需求与全模型汇总

四种模型的注意力结构与专家配置。本章选择一个稠密基线和三个不同规模的 MoE 模型。现在已经认识注意力、专家和层间连接，可以用相同的矩阵规则比较它们的资源需求。

表 2-A 四个模型的总体架构（文本主干）

架构参数	Qwen3-8B	Qwen3.6-35B-A3B	DeepSeek V4-Flash	Kimi K3
逻辑参数总数，约	8.19B	34.66B	284.33B	2.78T
主干层数 L	36	40	43	93
主干隐藏维度 d	4096	2048	4096	7168
注意力组成	36 GQA	30 线性 + 10 全注意力	2 窗口 + 21 CSA + 20 HCA	69 KDA + 24 MLA
前馈组成	稠密 SwiGLU	40 层 MoE	43 层 MoE	首层稠密 + 92 层 MoE
每 MoE 层路由专家数 E	—	256	256	896
每 token 选择数 k	—	8	6	16
路由专家计算宽度 d_e	—	2048	4096	3584
稠密 FFN / 路由专家中间宽度 f	12288	512	2048	3072
共享专家中间宽度	—	512	2048	合计 6144
跨层连接	普通残差	普通残差	mHC, 4 路	AttnRes

表中列出四个模型的文本主干。参数规模的后缀 B、T 分别表示十亿和万亿。Kimi K3 将不同的前馈结构用于不同层：首层是中间维度为 33792 的稠密 FFN，后续层则使用中间维度为 3072 的路由专家。稠密层让每个 token 经过同一组宽矩阵，MoE 层则让每个 token 使用多组较窄的专家矩阵。

先观察三个关系。Qwen3.6 的总参数比 Qwen3-8B 多，主干宽度却只有一半：它把大量参数放进可选择的专家，每个 token 只调用其中八个。Qwen3-8B 与 DeepSeek V4-Flash 的主干同为 4096 维，DeepSeek V4-Flash 通过每层 256 个专家形成更大的参数集合。Kimi K3 又将专家计算移到 3584 维潜空间，使主干宽度与专家输入宽度分开。这些差异解释了为什么“保存多大的模型”和“一个 token 做多少工作”需要分别计算。表 2-1、2-2、2-4、2-5 将逐个展开这四个模型，DeepSeek-V3 的表 2-3 用作 MLA 的历史参照。

下面把各类层的工作按数量累加，再加入模型入口与输出接口，得到整模型的需求。

前面已经可以计算一种层的参数、一次调用的运算和一条请求的状态。整模型只需把这些项按各类模型层的实际数量加权求和：

$$M_W = b_W \sum_g L_g N_g + M_{\text{embedding}} + M_{\text{head}}, \quad F_{\text{model}} = \sum_g L_g F_g + F_{\text{head}}.$$

g 表示模型层的类别，同一类层采用相同的注意力、前馈网络与连接结构； L_g 为第 g 类层的数量。MoE 的

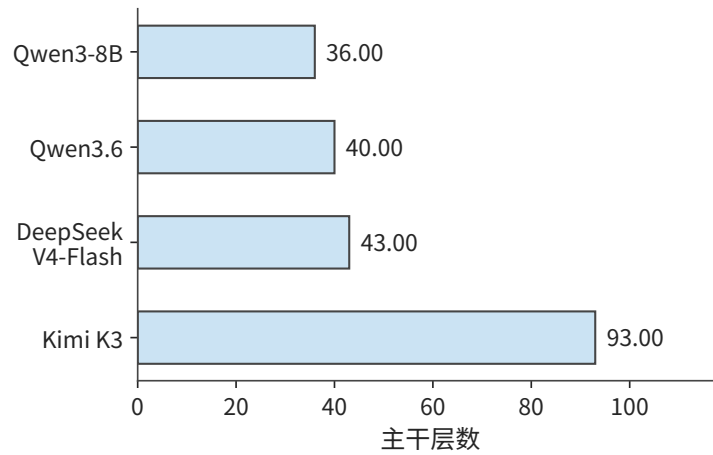


图 2-25 四模型的主干层数，使用同一线性尺度。层数决定同类工作沿网络深度重复多少次。

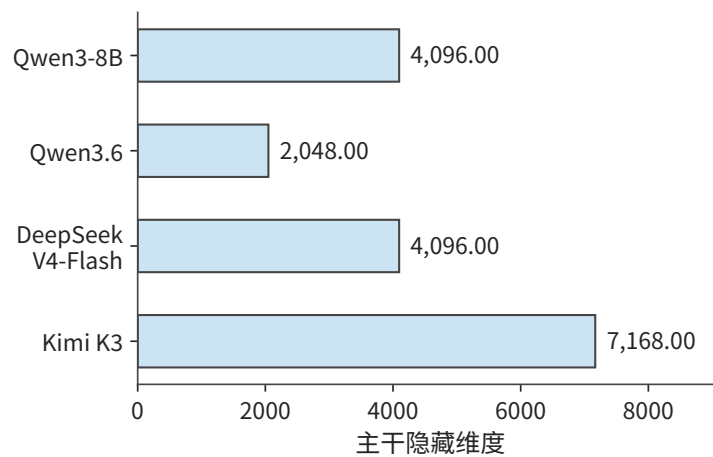


图 2-26 同一线性尺度下的主干隐藏维度。每个位置的特征宽度决定投影的输入或输出尺寸。

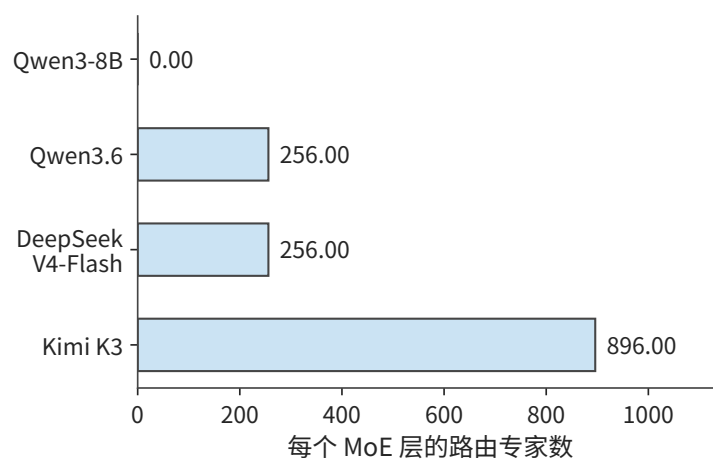


图 2-27 每个 MoE 层保存的路由专家数量。Qwen3-8B 使用稠密前馈层，其路由专家数为零；每个 token 实际选中数见表 2-A。

N_g 包含全部专家， F_g 则使用各专家实际接收的行数。表 2-1 已建立 Qwen3-8B 的稠密基线；章末查阅表按同样方式展开 Qwen3.6、DeepSeek-V3、DeepSeek V4-Flash 和 Kimi K3，方便读者查明总量中的每一项来自哪个矩阵。

阅读时可以按数据流走一遍：输入进入哪类注意力层，随后经过哪类前馈网络，最终如何得到词表分数。每张表按注意力、前馈与连接、输出接口分组；“层 / 调用数”给出整模型累计倍数，“矩阵 FLOPs”仍是该行对应操作执行一次的浮点运算量。先确定这一层采用的注意力、前馈网络与连接结构，再累加该层的注意力、前馈与连接工作，最后加入嵌入查行、末端归一化和词表头。

将同样的累加规则用于其他模型，可以逐项列出它们的计算分支。对 Qwen3-8B，表中包含嵌入、36 层注意力与 FFN、残差和归一化、最终输出头；对 Qwen3.6，还需分别计算线性注意力层与完整注意力层，并列路由与共享专家。对 DeepSeek V4-Flash，还要加入窗口、压缩、索引和 mHC；对 Kimi K3，则包括 KDA、MLA、潜空间投影、共享专家与 AttnRes。MTP 与视觉模块按本次执行范围单独加入。

模型分支	上下文状态	权重与附加计算	合计时最容易遗漏的部分
Qwen3-8B	GQA KV	稠密 SwiGLU、输出头	非矩阵算术、输出头位置范围
Qwen3.6	全局 KV + 递推与卷积状态	路由 / 共享专家	线性层保存固定大小的递推状态
DeepSeek-V3	MLA 潜变量或展开 KV，随执行路径变化	前三层稠密 FFN、后续 MoE 与共享专家	上投影执行位置、两种缓存路径的区别
DeepSeek V4-Flash	窗口 + 压缩 + 索引 + 缓冲	专家、压缩器、mHC	索引扫描、块边界更新、共享分支
Kimi K3	MLA + KDA + 卷积	潜空间专家、AttnRes	展开路径、跨层临时状态、首层差异

先依次比较完整权重、单步计算量和上下文状态大小，再通过各模型的分项表解释差异。Qwen3.6 的权重比 Qwen3-8B 多，单步计算和上下文状态却更少；DeepSeek V4-Flash 的专家权重继续增加，上下文压缩又使 8K 状态进一步减小。后面的表将说明这些差异分别来自哪些矩阵和状态。

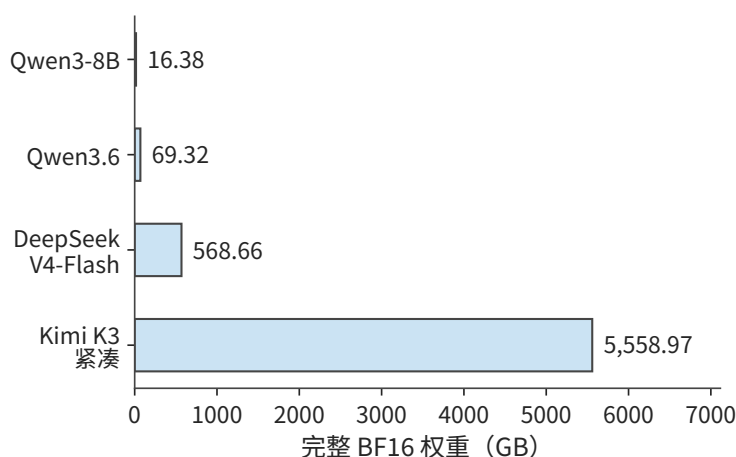


图 2-28 四模型完整权重统一用 BF16 表示时的容量，使用同一线性尺度。这里累计全部参数，包括每次仅选中部分的路由专家。

四个模型的逐矩阵展开见本章末的模型矩阵查阅表。表 2-2 至表 2-5 列出每个模块的输入输出尺寸、权重形状、单次运算量与层数。下面先用这些分项的合计解释资源差异。

将矩阵尺寸相乘并按层数累计，DeepSeek V4-Flash 主干约有 2843 亿个逻辑参数，统一 BF16 表示约为

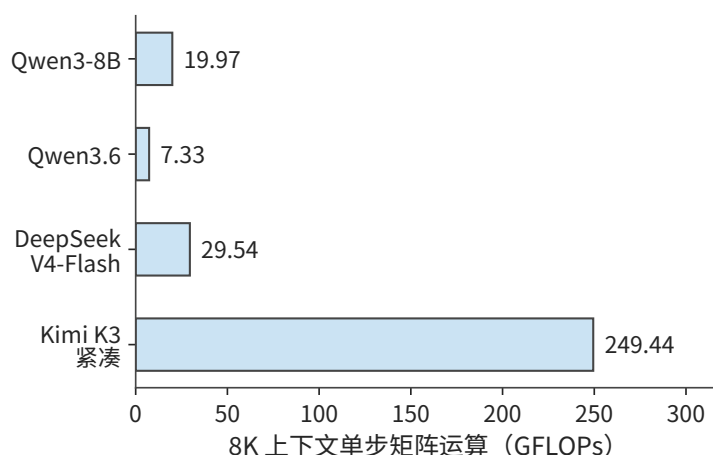


图 2-29 相同单请求、已有 8K 上下文时的单步矩阵运算量。按各模型在表 2-C 说明的执行路径累计，Kimi K3 采用紧凑 MLA。

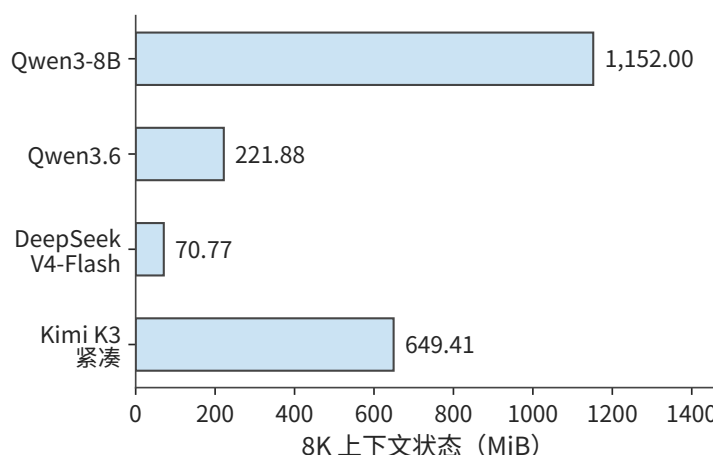


图 2-30 相同 8K 上下文下的每请求状态。上下文表示采用 BF16，递推矩阵和压缩缓冲采用相应实现的精度；各项合计对应表 2-C。

568.7 GB。发布的模型检查点（保存参数的文件集合）则采用混合存储格式，主干数据量约为 156.0 GB。前者回答结构包含多少参数，后者回答这些参数在文件中占多少字节；格式转换把二者连接起来。

Kimi K3 文本主干约有 2.78 万亿个参数，绝大多数位于 92 个 MoE 层的路由专家中。每个 token 在每层只选择 16 个专家，因此完整模型的存储容量与单 token 的计算量取决于不同的因素：前者由全部专家决定，后者由选中的专家及共同执行的注意力、共享专家和投影决定。²¹

下面将各模型的参数划分为互不重叠的五类：嵌入与输出、注意力投影、稠密或共享 FFN、路由专家、其余组件。将各类参数相加，就得到模型总参数量。这种分解能直观地展示参数分布，也为解释下一张计算与状态表提供依据。

²¹Kimi K3 模型配置说明；Kimi K3 前向计算。

表 2-B 参数由哪些部分构成（单位：十亿个逻辑参数）

模型	嵌入与输出头	注意力投影	稠密 / 共享		其他	合计
			FFN	路由专家		
Qwen3-8B	1.245	1.510	5.436	0.000	< 0.001	8.191
Qwen3.6	1.017	1.283	0.126	32.212	0.022	34.661
DeepSeek V4-Flash	1.059	5.085	1.082	277.025	0.081	284.332
Kimi K3（紧凑 MLA）	2.349	36.180	12.882	2722.741	5.333	2779.484

其他包含 router、潜空间进出投影、归一化、卷积和连接等参数；注意力投影包括 DeepSeek V4-Flash 的压缩与索引投影。Qwen3-8B 的主要参数集中在每个 token 都执行的稠密 FFN，三个 MoE 模型则将大部分参数放进路由专家。Qwen3.6 的路由专家占约 93%，DeepSeek V4-Flash 与 Kimi K3 约为 97%—98%。因此，它们的总容量主要由专家集合决定，单 token 的专家运算量则取决于每层选中的专家。

表 2-C 相同输入条件下的计算与状态

模型	统一 BF16 权重 (GB)	8K prefill (TFLOPs)	8K 上下文单步 decode (GFLOPs)	8K 状态 (MiB)	单 token 选中路由专家的 BF16 权重 (GiB)
Qwen3-8B	16.38	133.59	19.97	1152.00	—
Qwen3.6	69.32	52.09	7.33	221.88	1.875
DeepSeek V4-Flash	568.66	231.13	29.54	70.77	12.094
Kimi K3（紧凑 MLA）	5558.97	1863.46	249.44	649.41	90.562

两种调用均取 $B = 1$ ，prefill 为 $S = 0, P = 8192$ ，decode 为 $S = 8192, P = 1$ ，词表头均只处理末位置。状态列列出保存 8192 个上下文位置所需的空間：上下文表示采用 BF16，递推状态和压缩缓冲按相应实现采用 FP32。最后一列将一个 token 在各层选中的路由专家权重相加，再按 BF16 换算为字节数。由此可以分别比较完整模型占用的空間、一次调用的运算量，以及请求上下文占用的空間。

各模型按自己的执行路径累计：Qwen3-8B 为有效因果注意力，Qwen3.6 为逐项执行的矩形注意力实现（eager）与块式线性分支，DeepSeek V4-Flash 为有效主注意力及参考索引，Kimi K3 为紧凑 MLA 与块式 KDA。Qwen3.6 保存的权重约为 Qwen3-8B 的四倍，单步矩阵运算量却更少；DeepSeek V4-Flash 权重继续扩大，8K 状态反而因压缩与选择缩小；Kimi K3 拥有更多专家，隐藏维度更大、层数也更多，因此需要保存更多权重，完成更多运算。这些差异说明，总参数量、单次运算量和上下文状态大小需要根据各自对应的结构分别计算。

由上下文状态大小推算单步读取量。 Qwen3-8B 每步使用 1152 MiB 旧 KV；DeepSeek V4-Flash 的主注意力读取与索引扫描合计 27.625 MiB，并更新压缩器。Kimi K3 紧凑 MLA 使用 216 MiB 上下文状态，414 MiB 递推矩阵还要一读一写，读写量合计 828 MiB；Qwen3.6 对应 160 MiB 全局注意力的 KV 缓存和 120 MiB 递推矩阵读写。递推状态每步都要读出并更新，上下文表示则要供当前查询使用。因此，即使状态大小固定，每步也仍有相应的读写开销。

量化编码与元数据共同决定权重容量。 统一按 BF16 计算时，每个参数占 2 字节，权重大小可以直接反映参数量的差异。实际发布的权重文件（checkpoint，即模型参数的保存快照）使用各自的存储格式，DeepSeek V4-Flash 主干约为 156.02 GB，Kimi K3 文本约为 1559.97 GB。文件大小由低位宽矩阵、采用较高精度的参数以及格式元数据共同决定。模型加载后，还需要为格式转换和计算分配空間。第五章将分析存储格式与运行时

实现如何影响显存占用。

表 2-B 的参数组成解释了权重容量，表 2-C 则展示这些参数怎样参与一次调用。二者结合起来，可以区分“模型能容纳多少参数”与“处理一个 token 要使用多少资源”。²²

上下文从 8K 增至 200K 时的运算量与状态增长。长文档问答和多轮任务往往需要保留远多于 8K 的上下文。下面保持单请求、一次新增一个 token 和末位置词表头不变，只把调用前的上下文从 8192 增至 204800 个位置；本例的 K 均指 1024，所以上下文长度增至 25 倍。

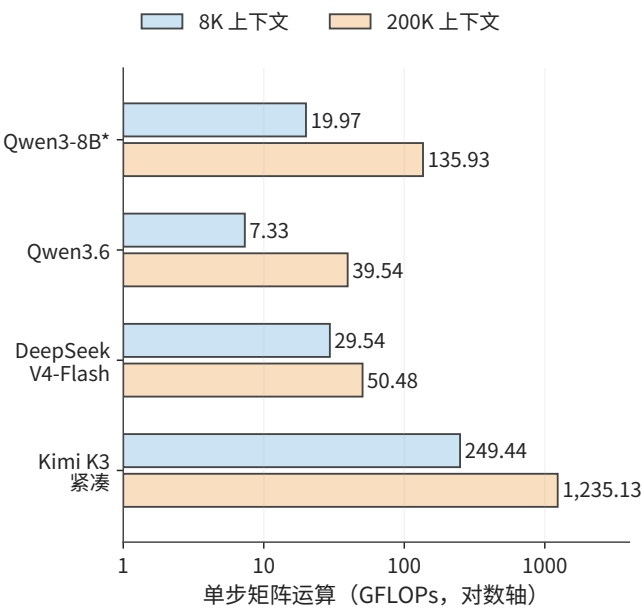


图 2-31 已有 8K / 200K 上下文后再处理一个 token 的矩阵运算量。蓝、橙分别对应两种上下文长度，横轴为对数刻度，柱端标出 GFLOPs。沿用表 2-C 的 decode 路径，Kimi K3 的 KDA 使用递推、MLA 使用紧凑表示。星号表示 Qwen3-8B 的 200K 条件超过本章固定配置的未缩放上下文上限，仅按原结构外推。

表 2-D 将上下文交互从整模型运算中提取出来，累计当前查询的评分与值汇总，DeepSeek V4-Flash 再加索引点积扫描。状态列包含调用前的上下文与固定状态，精度沿用表 2-C。把交互项与图 2-31 的整模型矩阵运算对照，就能看出上下文增长带来的新增工作。²³

表 2-D 长上下文下的交互运算与状态容量

模型	8K 上下文交互 (GFLOPs)	200K 上下文交互 (GFLOPs)	8K 状态 (GiB)	200K 状态 (GiB)
Qwen3-8B (200K 为 外推)	4.83	120.80	1.125	28.125
Qwen3.6	1.34	33.55	0.217	3.967
DeepSeek V4-Flash	3.00	23.94	0.069	1.329
Kimi K3 (紧凑 MLA)	41.08	1026.77	0.634	5.697

DeepSeek V4-Flash 的收益可以从条目数直接看出。200K 上下文下，一个 CSA 层保存 51200 个压缩条目，索引扫描这些条目后，主注意力最多使用其中 512 个，再加 128 个窗口位置；一个 HCA 层使用 1600 个粗粒度

²²四模型比较数据；生成程序；Qwen3.6 末位置输出头计算。
²³8K / 200K 比较数据；复算程序。程序核对 8K 数值与表 2-C 一致，并独立检查 DeepSeek V4-Flash 的窗口、CSA、HCA 与索引公式。

条目，再加窗口。与 8K 相比，CSA 的主注意力访问上限保持不变，HCA 和索引扫描继续增长，因此整模型单步矩阵运算只从 29.54 增至 50.48 GFLOPs，约为 1.71 倍。

为单独比较上下文交互的矩阵工作，固定 DeepSeek V4-Flash 的 43 层、64 个查询头和 512 维头宽，假设每层对全部 204801 个位置执行完整评分与汇总。这项矩阵工作为 $43 \times 4 \times 64 \times 512 \times 204801$ ，约 1154.28 GFLOPs；实际窗口、压缩上下文及索引扫描合计约 23.94 GFLOPs，相差约 48.2 倍。窗口、压缩和索引选择共同减少了参与评分与汇总的位置数。

Kimi K3 则通过两种机制降低长上下文的负担。69 个 KDA 层把过去输入汇入固定大小的状态，只有 24 个 MLA 层继续逐位置访问上下文。若仅比较相同头数和紧凑宽度下的上下文交互项，24 层需要的工作是 93 层全部采用这类 MLA 时的 24/93，约 25.8%；KDA 自身仍需要投影与递推更新。与此同时，24 个 MLA 层在 200K 下保存的紧凑上下文表示约为 5.273 GiB，若保存展开后的各头 K/V，则需 281.25 GiB，相差约 53.3 倍。紧凑表示减少了保存的数据，而潜空间点积的宽度决定每次交互的运算量。

因此，Kimi K3 的整模型单步矩阵运算仍从 249.44 增至 1235.13 GFLOPs：长上下文最终会放大剩余 MLA 层的工作。评估上下文压缩时，应同时看保存了多少状态、每步访问多少上下文，以及为压缩和更新状态付出多少运算。

2.6.2 参数规模、数值精度与容量约束

分项计算给出了模型权重和每条请求状态的大小。现在把它们放进同一张显存容量表，就可以判断一台设备能同时容纳多少条请求，以及量化权重能腾出多少空间。

对一台设备，先从总容量中扣除权重、工作区和其他固定预留空间，剩余空间才能放请求状态。对普通等长、独立 KV 的简化情形，可写为：

$$B_{\max} = \left\lfloor \frac{M_{\text{device}} - M_{\text{weight}} - M_{\text{workspace}}}{M_{\text{state/request}}} \right\rfloor$$

分子是扣除固定驻留之后可用于请求的空间，分母是每条请求的状态。每增加一条请求，都要为它分配相应的状态空间；剩余空间放不下完整的一份状态时，就无法再增加请求，因此结果要向下取整。递推与卷积状态同样属于每请求数据，与 KV 一起计入分母。

以 24 GB 显存运行 Qwen3-8B BF16 为例，权重占 16,381,470,720 字节，工作区预留 2 GiB，每条请求的 8K 上下文占 1.125 GiB。可容纳请求数为

$$B_{\max} = \left\lfloor \frac{24 \times 10^9 - 16,381,470,720 - 2 \times 2^{30}}{1.125 \times 2^{30}} \right\rfloor = 4.$$

上下文变为 16K 时，每条请求占 2.25 GiB，同一空间只能放两条；变为 4K 时，则能放九条。整数跳变来自剩余空间必须足以容纳完整的一条状态，而不是只够存放其中一部分。

保留多少历史由此成为一个同时影响应用和系统的决定。应用多保留一段材料，模型就能继续参考它；服务需要为每条请求分配更多状态空间。前面的 GQA、MLA 和递推表示进一步改变了保存这些信息的方式。模型结构、上下文组织与并发容量可以放在同一份预算里比较，任务检查则确定哪些方案能够提供所需的回答质量。

再看 DeepSeek-R1-Distill-Llama-70B，其参数量约为 705.5 亿。其 BF16 权重为 141.107 GB，每条 8192-token BF16 KV 为 2.5 GiB。在本例采用的分组量化方案中，让嵌入、输出头和归一化等部分继续使用较高精

度，并计入 scale 与打包尾部后，8-bit 方案约为 73.726 GB，4-bit 方案约为 39.500 GB。

4-bit 方案的 39.500 GB 包括低位宽矩阵、仍采用高精度的参数，以及 scale 与打包开销，比将全部参数一律按半字节计算更大。显存容量为 80 GB（十进制）、固定预留 2 GiB 时，8-bit 方案可放 1 条 8K 请求，4-bit 方案可放 14 条。上下文从 8K 增至 32K 时，每请求 KV 增至四倍，4-bit 方案可同时容纳的请求数降到 3 条。压缩权重腾出的空间可以保存更多请求的上下文，但每条上下文越长，能同时容纳的请求就越少。²⁴

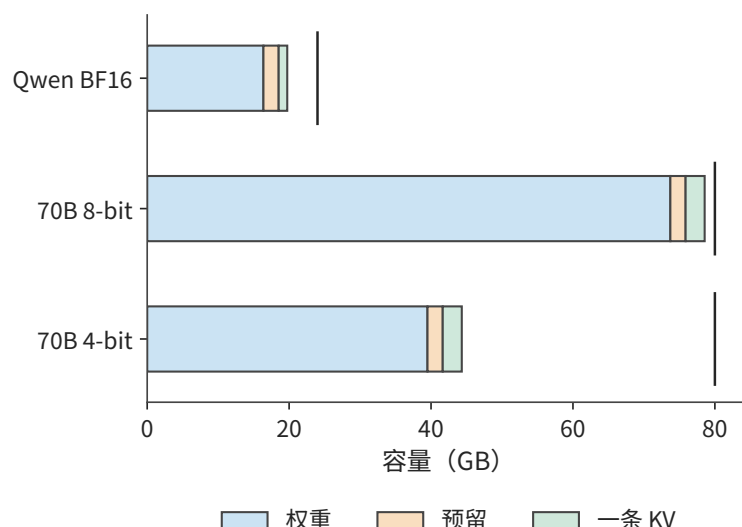


图 2-32 权重、固定预留和一条请求 KV 的逐项容量。短竖线标出各设备容量；KV 采用 BF16、上下文长度 8192，固定预留为 2 GiB。

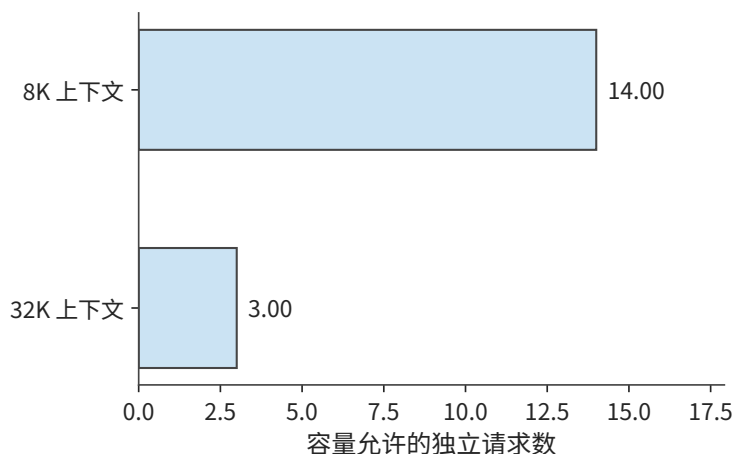


图 2-33 70B 的 4-bit 方案在同一张 80 GB 设备上，上下文长度从 8K 增至 32K 时可同时容纳的独立请求数。每请求状态增加，使剩余空间容纳的请求数减少。

同样方法也能分析更大的部署。235B 的 BF16 权重约 470 GB，超过八张 24 GB 卡合计的 192 GB，而八张 80 GB 卡合计为 640 GB。后者为状态、工作区与复制留下约 170 GB 总余量；第六章将按具体分工把这笔总量分配到每张卡，得到逐卡预算。

容量决定一个实例至少需要多少设备，请求长度与并发数则决定实例需要处理多少工作。更低位宽释放空间，较短上下文允许更多请求同时驻留，批内复用则改变这些请求的执行成本。因此，选择部署方案时需要依次考虑容量、访问量与运算量，第三章将进一步加入请求的到达与时限。

²⁴8K 容量表；32K 对照。

练习 2-7 (核心): 由显存容量决定可容纳请求数

显存为 24 GB，工作区预留 2 GiB，Qwen3-8B BF16 权重采用表中的精确值。分别求 4K、8K、16K 上下文最多可同时容纳多少条请求；在 8K 情形下，将工作区预留增至 3 GiB，重新计算。再使用表中的 70B 4-bit 权重 39.500 GB，将可用容量改为 48 GB、预留仍为 2 GiB，分别求 8K 与 32K 上下文的请求数。先比较权重、工作区与 KV 的容量需求，再用剩余显存计算可容纳的请求数。

还可以反过来问：给定设备和服务目标，留给模型多少空间？设每卡可用容量为 C ，固定预留工作区为 M_0 ，每条请求的 KV 为 K ，并发为 B ，每参数占 b_w 字节。忽略其他驻留对象，单卡容量给出的参数上界为

$$N_{\max} = \frac{C - M_0 - BK}{b_w}.$$

先固定服务状态，计算留给权重的空间。取 24 GB 容量、2 GiB 工作区、4 条各 8192 位置的请求，每请求 KV 沿用 Qwen3-8B 的 1.125 GiB。BF16 每参数 2 字节，剩余空间最多容纳约 85.1 亿参数。²⁵图中先扣除服务状态，再划出权重预算。

改变模型结构时，权重预算与状态预算会一起变化。增加层数或 KV 头数会增大每请求的 K ，留给权重的空间随之减少；跨设备部署则扩大总容量，同时增加通信。Llama 3 报告中的 405B BF16 推理使用两台、共 16 张 H100，便跨越了单机容量边界。²⁶模型规模、状态结构与设备数量因此需要共同选择。

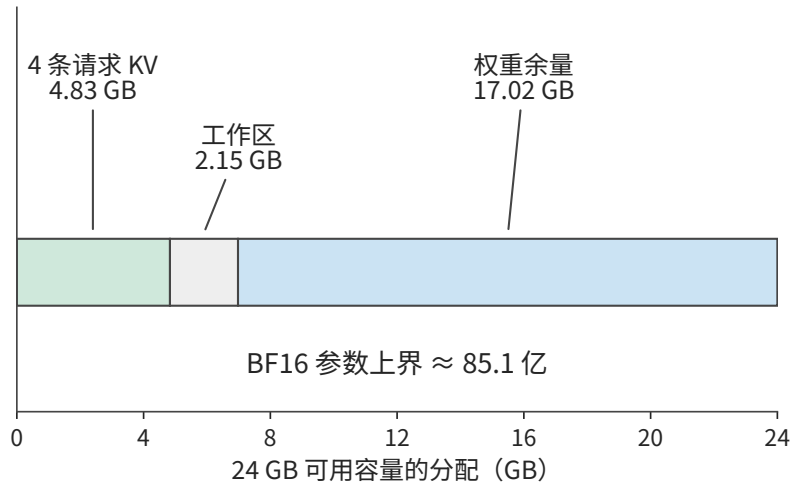


图 2-34 从设备反推权重预算。24 GB 中先为 4 条 8K 请求的 KV 和 2 GiB 工作区预留空间，余量给出 BF16 参数上界；图中容量均为十进制 GB。

2.6.3 不同请求条件下的模型比较

最后把容量、调用链和任务质量合在一起。先根据设备容量排除无法放下的配置，再沿每个模型的实际计算方式累计请求工作，最后检查模型能否完成目标任务。

比较完整模型时，可以从两种输入出发。固定 S, P, G, B ，得到相同调用长度下的资源需求，便于观察结构差异；固定文本和任务，则先经各模型的分词器得到输入长度，再检查输出。这两种对照分别回答“同样多的位置怎样计算”和“同一个任务需要怎样执行”。

²⁵本章条件比较的逐项复算与计算程序。

²⁶MQA 原文摘要；GQA 原文摘要；Llama 3 报告第 6.1 节。容量示例是本书按固定服务状态推导的教学预算，不是这些模型的历史设计记录。

先比较相同输入、输出长度下的资源需求。取 $S = 0, P = 128, G = 4, B = 1$ ，模型执行一次 prefill 和三次 decode，最终保存 131 个已处理位置。将这些调用的矩阵运算量相加，得到下表。²⁷

模型	完整请求矩阵运算量 (TFLOPs)	本次计算采用的实现方式
Qwen3-8B	1.83	有效因果矩阵运算量，最后位置输出头
DeepSeek V4-Flash	3.40	主干有效注意力 / 专家工作，参考索引计算
DeepSeek V4-Pro	12.72	独立 Pro 配置，按该配置中各类注意力层的数量和条目选择上限计算
Kimi K3	27.13	展开 MLA 与块式 KDA 算法

这张表中的 Kimi K3 使用展开 MLA：上下文先恢复为各头的 K 、 V ，再执行交互。前面的紧凑 MLA 则把上投影移到查询和汇总一侧。完整请求沿同一条选定路径累计 prefill 与三次 decode，上下文从 128 增到 131；这一路径的矩阵形状与状态表示共同决定各项运算量。

从一次调用扩展到完整请求之后，输入和输出的作用更加清楚。增加 P ，主要增加一次输入处理的工作量，以及后续解码开始时的上下文长度；增加 G ，则反复执行投影、专家与输出头，并多次读取上下文。长输入、短输出与短输入、长输出由此形成不同的资源组成，第三章将分析这两类请求持续到达时的系统负载。

练习 2-8 [延伸]：从多轮输入重建调用

一轮输入总长 1443 个 token，其中前 1392 个命中缓存，返回 128 个 token。求 S, P, G, n_d 的值、请求结束时的上下文长度，以及本轮新增 KV 的大小。比较完全重算与复用缓存时，投影行数、有效因果位置对数分别减少多少。若在第 501 个输入位置修改了内容，重新求最长可复用前缀与新输入长度。

固定文本时，不同分词器会产生不同的输入长度。例如，在八道短语检索题中，Qwen3-8B 与 DeepSeek V4-Flash 都正确找到了目标短语，但短文在 Qwen 中对应 523 或 524 个 token，在 DeepSeek V4-Flash 中对应 501 个；较长文本分别对应 2121 或 2122 个和 2036 个。相同文本保持了任务内容，模型各自的 token 数则决定了实际输入形状。

因此，比较模型需要同时回答两个问题：能否完成同一任务，以及完成任务需要多少资源。先按相同标准判断答案，再将各自的输入长度代入模型表，才能把结构差异与实际用途联系起来。²⁸

练习 2-9 [延伸]：根据容量、计算需求与任务质量选择模型

使用四模型资源表，为每个模型分别列出完整权重、8K 状态、单步 decode 与 8K prefill 的需求。设单卡显存为 80 GB，预留 2 GiB 工作区，并采用 BF16 权重。先判断哪些模型能完整放入单卡；再把问题改为多卡部署，说明还需从后续章节获得哪些时间与通信参数。将短请求的 G 改为 1，检验调用次数。最后为一个短语检索任务写出质量标准，说明同 token 数比较与同文本比较分别回答什么问题。

本章从计算图出发，逐步算出了完整请求的资源需求。注意力决定如何保存和访问上下文，MoE 决定每次选择哪些权重，网络深度与连接方式决定计算顺序和临时存储。下一章将加入请求长度分布、到达时间、工具调用、训练与 RL，分析这些计算如何随时间累积成系统负载。

²⁷ 四模型完整请求计算。

²⁸ 配对检索实验。Qwen 使用 BF16 / vLLM；DeepSeek V4-Flash 使用 MXFP4 专家、FP8 KV / SGLang，并将部分权重放在 CPU 内存。

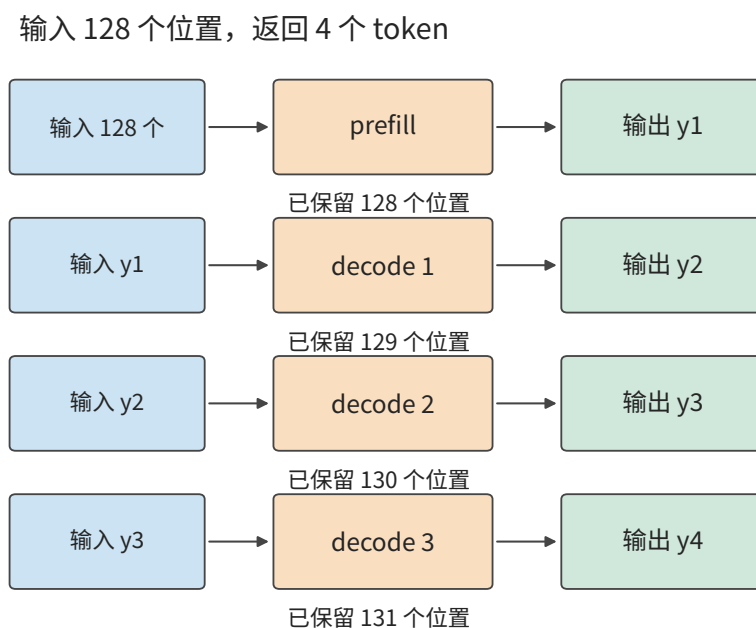


图 2-35 生成四个输出的调用顺序。prefill 处理 128 个输入并产生首输出，随后三次 decode 各将前一输出送回模型；最终保留 131 个位置。

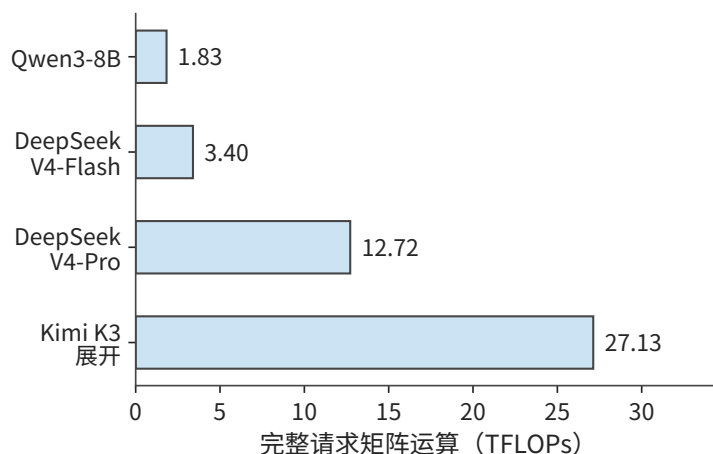


图 2-36 同一 128 输入、四输出请求的矩阵运算量。模型执行范围与缓存路径见本节题设；这是逐调用累计的计算量。

谬误与陷阱

参数更多，每 token 计算就一定更多。总参数决定完整模型的容量；MoE 的选中专家、注意力机制的类型和调用形状决定当前工作。对照表 2-A 与资源表，分别解释参数数量和计算量由哪些因素决定。

KV 缩小到原来的四分之一，注意力计算量就降到原来的四分之一。GQA 共享上下文表示，查询头仍分别评分；MLA 改变上下文宽度，也改变上投影的执行位置。先找被改变的矩阵与状态，再计算其余工作。

状态不随上下文增长，状态访问就可以忽略。固定递推矩阵仍需更新；prefill 还可能产生临时块状态。容量、累计访问与峰值工作区分别回答不同问题。

同 token 数就是公平的模型能力比较。输入和输出的 token 数适合比较资源结构；同文本任务还需固定评分、质量与执行条件，并允许不同分词器产生不同长度。

本章练习与复现

练习 2-2、2-5、2-7 为核心；其余练习用于分析特殊情况，并将方法应用于其他模型。先独立完成预测与推导，再用配套计算核对。计算命令在仓库根目录运行，完整变体参见第二章扩写资料。

对应内容	本地复算入口
序列依赖与缓存等价性	<code>python3 calculations/calc.py sequence-dependencies --format md</code>
Qwen3.6 前向与专家	<code>python3 calculations/calc.py qwen36-forward --format md</code>
DeepSeek V4-Flash 状态分解	<code>python3 calculations/calc.py state --model deepseek-v4-flash --length 8192</code>
Kimi K3 状态与块式执行	<code>python3 calculations/calc.py k3-kda-chunk --tokens 8192</code>
专家粒度	<code>python3 calculations/calc.py qwen235-expert-granularity --format md</code>
单次 MTP	<code>python3 calculations/calc.py v4-mtp-forward --format md</code>
已知轨迹字段代入	<code>python3 calculations/calc.py trace-resource-bridge --format md</code>
四模型请求比较	<code>python3 calculations/calc.py request-model-comparison --format md</code>

本章配图、计算数据与重建方法见配图目录。下列资料介绍各模型的结构与设计。

模型矩阵查阅表

这组表用于从整模型合计追溯到具体矩阵。查阅时先确定模型与执行分支，再用单次矩阵 FLOPs 乘层数；路由专家还要按实际接收的输入行数累计。表 2-1 的 Qwen3-8B 基线见 2.1.4。

表 2-2 Qwen3.6-35B-A3B：混合注意力与 MoE 各模块的资源消耗计算

主干共 40 层：10 层选完整注意力分支，30 层选线性注意力分支；两类层随后都执行 MoE。每 token 从 256 个路由专家中选 8 个，另有一个共享专家。线性分支的 QKV 输出拆为 2048、2048、4096 维；其 Q 、 K 会按值头的分组关系复用。

模型入口

		权重矩阵（输入宽 $\times$ 输出宽）		
模块及作用	输入 $\rightarrow$ 输出（行数 $\times$ 宽度）	出宽	矩阵 FLOPs	层 / 调用数
词嵌入查行	m 个 ID $\rightarrow m \times 2048$	查表 248320×2048	0；查行访问另计	1

完整注意力分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
完整注意力: Q 与输出门控	$m \times 2048 \rightarrow m \times 8192$	2048×8192	$2 \times m \times 2048 \times 8192$	10
完整注意力: K 、 V	$m \times 2048 \rightarrow m \times 512$	2048×512 , 共 2 个	$2 \times 2 \times m \times 2048 \times 512$	10
完整注意力: 上下文交互	16 个 Q 头、2 个 KV 头, 头宽 256	无新增权重	$4 \times 16 \times 256 \times N_{\text{pair}}$	10
完整注意力: 输出投影	$m \times 4096 \rightarrow m \times 2048$	4096×2048	$2 \times m \times 4096 \times 2048$	10

线性注意力分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
线性注意力: QKV 联合投影	$m \times 2048 \rightarrow m \times 8192$	2048×8192	$2 \times m \times 2048 \times 8192$	30
线性注意力: 输出门控 z	$m \times 2048 \rightarrow m \times 4096$	2048×4096	$2 \times m \times 2048 \times 4096$	30
线性注意力: a 、 b 控制量	$m \times 2048 \rightarrow m \times 32$	2048×32 , 共 2 个	$2 \times 2 \times m \times 2048 \times 32$	30
线性注意力: 卷积与 delta 递推	Q, K 各 16×128 ; V 为 32×128	卷积: 8192 通道、核宽 4	递推为 $O(m \times 32 \times 128^2)$; 块式算法另计	30
线性注意力: 输出投影	$m \times 4096 \rightarrow m \times 2048$	4096×2048	$2 \times m \times 4096 \times 2048$	30

各层共同的专家分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
每层: 路由评分	$m \times 2048 \rightarrow m \times 256$	2048×256	$2 \times m \times 2048 \times 256$	40
路由专家 e : 在选中分支变换	$t_e \times 2048 \rightarrow t_e \times 512 \rightarrow t_e \times 2048$	2048×512 两个; 512×2048 一个	$6 \times t_e \times 2048 \times 512$	40
共享专家: 所有 token 的共同变换	$m \times 2048 \rightarrow m \times 512 \rightarrow m \times 2048$	2048×512 两个; 512×2048 一个	$6 \times m \times 2048 \times 512$	40
共享专家门控	$m \times 2048 \rightarrow m \times 1$	2048×1	$2 \times m \times 2048 \times 1$	40

输出接口

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
词表头 (末端一次)	$m_{\text{out}} \times 2048 \rightarrow m_{\text{out}} \times 248320$	2048×248320	$2 \times m_{\text{out}} \times 2048 \times 248320$	1

累计时, 用各行公式乘最后一列的层数; 路由专家再对实际接收行数 t_e 求和。末端归一化执行一次, 层内归一化、激活、位置处理和状态更新沿相应分支执行。

读这张表时, 先根据这一层使用线性注意力还是完整注意力, 选择对应的一组表格行, 再加 MoE 的共同部分。线性注意力决定上下文如何保存, MoE 决定当前 token 使用哪些权重; 两种机制可以同时存在。A3B 描述

名义激活规模，全部专家仍共同构成常驻模型。²⁹

表 2-3 DeepSeek-V3: MLA 与稠密/专家前馈网络

DeepSeek-V3 共 61 层，前三层使用稠密 FFN，后 58 层每 token 选择 8 个路由专家并执行一个共享专家。下面沿展开 MLA 计算：每个位置的潜变量经上投影形成各头的 K 、 V ，64 维位置表示与 K 拼接。查询与键共同参与位置相关的评分， V 则用于汇总内容。

模型入口

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词嵌入查行	m 个 ID → $m \times 7168$	查表 129280×7168	0；查行访问另计	1

MLA 注意力

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
查询下投影：形成低秩查询	$m \times 7168 \rightarrow m \times 1536$	7168×1536	$2 \times m \times 7168 \times 1536$	61
查询上投影：展开 128 个头	$m \times 1536 \rightarrow m \times 24576$	1536×24576	$2 \times m \times 1536 \times 24576$	61
KV 下投影：512 维潜变量及 64 维位置分支	$m \times 7168 \rightarrow m \times 576$	7168×576	$2 \times m \times 7168 \times 576$	61
KV 上投影：展开每头 K 、 V	$m \times 512 \rightarrow m \times 32768$	512×32768	$2 \times m \times 512 \times 32768$	61
展开 MLA：评分与值汇总	128 头；Q/K 宽 192， V 宽 128	无新增权重	$2 \times 128 \times (192 + 128)N_{\text{pair}}$	61
注意力输出投影	$m \times 16384 \rightarrow m \times 7168$	16384×7168	$2 \times m \times 16384 \times 7168$	61

前馈分支

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
前三层：稠密前馈网络	$m \times 7168 \rightarrow m \times 18432 \rightarrow m \times 7168$	7168×18432 两个； 18432×7168 一个	$6 \times m \times 7168 \times 18432$	3
后 58 层：路由评分	$m \times 7168 \rightarrow m \times 256$	7168×256	$2 \times m \times 7168 \times 256$	58
路由专家 e	$t_e \times 7168 \rightarrow t_e \times 2048 \rightarrow t_e \times 7168$	7168×2048 两个； 2048×7168 一个	$6 \times t_e \times 7168 \times 2048$	58
共享专家	$m \times 7168 \rightarrow m \times 2048 \rightarrow m \times 7168$	7168×2048 两个； 2048×7168 一个	$6 \times m \times 7168 \times 2048$	58

输出接口

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词表头（末端一次）	$m_{\text{out}} \times 7168 \rightarrow m_{\text{out}} \times 129280$	7168×129280	$2 \times m_{\text{out}} \times 7168 \times 129280$	1

²⁹Qwen3.6 前向记录。

累计时，用各行公式乘最后一列的层数；路由专家再对实际接收行数 t_e 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

V3 将两种节省资源的方法组合起来：MLA 用低秩潜变量表示上下文，MoE 让一个 token 只执行部分专家。DeepSeek V4-Flash 进一步改变上下文的组织方式，用窗口保存近期位置，再用不同压缩比例保存长期上下文。下面的模块表将这几类上下文分别展开。³⁰

表 2-4 DeepSeek V4-Flash：选择性上下文访问与专家计算

43 层包含 2 个纯窗口层、21 个 CSA 层和 20 个 HCA 层；每 token 选 6 个路由专家，另有一个共享专家。 A 是该层跨全部请求的主注意力有效位置对数， I 是索引实际扫描的位置对数，两者均不含头数。分组输出投影的八组处理八份不同切片。CSA 的压缩投影先产生包含重叠分支的 $1024 / 256$ 维表示，再分别形成 512 维的主注意力上下文表示和 128 维的索引条目。

模型入口

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词嵌入查行	m 个 ID → $m \times 4096$	查表 129280×4096	0；查行访问另计	1

所有层的注意力投影

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
所有层：查询下投影	$m \times 4096 \rightarrow m \times 1024$	4096×1024	$2 \times m \times 4096 \times 1024$	43
所有层：查询上投影	$m \times 1024 \rightarrow m \times 32768$	1024×32768	$2 \times m \times 1024 \times 32768$	43
所有层：共享 KV	$m \times 4096 \rightarrow m \times 512$	4096×512	$2 \times m \times 4096 \times 512$	43
主注意力：窗口及所选压缩上下文	64 头，每头 512 维	无新增权重	$4 \times 64 \times 512 \times A$	43
所有层：分组输出低秩投影	$m \times 4096 \rightarrow m \times 1024$	4096×1024 ，共 8 个	$8 \times 2 \times m \times 4096 \times 1024$	43
所有层：拼接后输出投影	$m \times 8192 \rightarrow m \times 4096$	8192×4096	$2 \times m \times 8192 \times 4096$	43

CSA 专有工作

模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
CSA：主注意力的上下文压缩与门控	$m \times 4096 \rightarrow m \times 1024$	4096×1024 ，共 2 个	$2 \times 2 \times m \times 4096 \times 1024$	21
CSA：索引查询	$m \times 1024 \rightarrow m \times 8192$	1024×8192	$2 \times m \times 1024 \times 8192$	21
CSA：索引头权重	$m \times 4096 \rightarrow m \times 64$	4096×64	$2 \times m \times 4096 \times 64$	21
CSA：索引压缩内容及门控	$m \times 4096 \rightarrow m \times 256$	4096×256 ，共 2 个	$2 \times 2 \times m \times 4096 \times 256$	21
CSA：索引扫描及选择	64 个索引头，每头 128 维	无新增权重	点积 $2 \times 64 \times 128 \times I$ ；加权与 top-k 另计	21

³⁰V3 前向计算。

HCA 专有工作				
模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
HCA：主注意力的上下文压缩与门控	$m \times 4096 \rightarrow m \times 512$	4096×512 ，共 2 个	$2 \times 2 \times m \times 4096 \times 512$	20

各层专家与连接				
模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
路由评分	$m \times 4096 \rightarrow m \times 256$	4096×256	$2 \times m \times 4096 \times 256$	43
路由专家 e	$t_e \times 4096 \rightarrow t_e \times 2048 \rightarrow t_e \times 4096$	4096×2048 两个； 2048×4096 一个	$6 \times t_e \times 4096 \times 2048$	43
共享专家	$m \times 4096 \rightarrow m \times 2048 \rightarrow m \times 4096$	4096×2048 两个； 2048×4096 一个	$6 \times m \times 4096 \times 2048$	43
mHC：残差汇合与混合	$m \times 4 \times 4096 \leftrightarrow m \times 4096$	混合投影与归一化参数	混合矩阵、标量与特殊函数独立累计	43

输出接口				
模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词表头（末端一次）	$m_{\text{out}} \times 4096 \rightarrow m_{\text{out}} \times 129280$	4096×129280	$2 \times m_{\text{out}} \times 4096 \times 129280$	1

累计时，用各行公式乘最后一列的层数；路由专家再对实际接收行数 t_e 求和。末端归一化执行一次，层内归一化、激活、位置处理和状态更新沿相应分支执行。

这张表解释了为什么“每步只选少量上下文”不等于总工作固定：主注意力使用 A ，索引扫描使用 I ，压缩投影则对新输入持续执行。这三类工作量的增长规律不同。³¹

表 2-5 Kimi K3：KDA、紧凑 MLA 与潜空间专家

93 层包含 69 个 KDA 层、24 个 MLA 层；首层使用稠密 FFN，其余层从 896 个路由专家中选 16 个，并执行合计宽度 $2 \times 3072 = 6144$ 的共享分支。表内 MLA 采用紧凑代数路径，96 路小投影分别使用每头参数。共享专家保持 7168 维主干空间，路由专家才进入 3584 维潜空间。

模型入口				
模块及作用	输入 → 输出（行数 × 宽度）	权重矩阵（输入宽 × 输出宽）	矩阵 FLOPs	层 / 调用数
词嵌入查行	m 个 ID $\rightarrow m \times 7168$	查表 163840×7168	0；查行访问另计	1

³¹DeepSeek V4-Flash 完整前向记录。

KDA 分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
KDA: Q 、 K 、 V	$m \times 7168 \rightarrow m \times 12288$	7168×12288 , 共 3 个	$3 \times 2 \times m \times 7168 \times 12288$	69
KDA: 衰减低秩投影 a	$m \times 7168 \rightarrow m \times 128$	7168×128	$2 \times m \times 7168 \times 128$	69
KDA: 衰减低秩投影 b	$m \times 128 \rightarrow m \times 12288$	128×12288	$2 \times m \times 128 \times 12288$	69
KDA: 更新门控	$m \times 7168 \rightarrow m \times 96$	7168×96	$2 \times m \times 7168 \times 96$	69
KDA: 输出门控	$m \times 7168 \rightarrow m \times 12288$	7168×12288	$2 \times m \times 7168 \times 12288$	69
KDA: 有限状态更新	96 个 128×128 状态矩阵	短卷积及衰减参数	递推 $O(m \times 96 \times 128^2)$; 块式实现另计	69
KDA: 输出投影	$m \times 12288 \rightarrow m \times 7168$	12288×7168	$2 \times m \times 12288 \times 7168$	69

MLA 分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
MLA: 查询下投影	$m \times 7168 \rightarrow m \times 1536$	7168×1536	$2 \times m \times 7168 \times 1536$	24
MLA: 查询上投影	$m \times 1536 \rightarrow m \times 18432$	1536×18432	$2 \times m \times 1536 \times 18432$	24
MLA: 潜变量及额外分支	$m \times 7168 \rightarrow m \times 576$	7168×576	$2 \times m \times 7168 \times 576$	24
MLA: 将键上投影吸收到查询侧	$m \times 128 \rightarrow m \times 512$	128×512 , 共 96 个	$96 \times 2 \times m \times 128 \times 512$	24
紧凑 MLA: 潜空间评分与汇总	96 头; 评分宽 576, 汇总宽 512	使用紧凑的上下文表示	$2 \times 96 \times (576 + 512)N_{\text{pair}}$	24
MLA: 汇总后恢复值表示	$m \times 512 \rightarrow m \times 128$	512×128 , 共 96 个	$96 \times 2 \times m \times 512 \times 128$	24
MLA: 输出门控	$m \times 7168 \rightarrow m \times 12288$	7168×12288	$2 \times m \times 7168 \times 12288$	24
MLA: 输出投影	$m \times 12288 \rightarrow m \times 7168$	12288×7168	$2 \times m \times 12288 \times 7168$	24

前馈与专家分支

模块及作用	输入 → 输出 (行数 × 宽度)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
首层: 稠密前馈网络	$m \times 7168 \rightarrow m \times 33792 \rightarrow m \times 7168$	7168×33792 两个; 33792×7168 一个	$6 \times m \times 7168 \times 33792$	1
其余层: 路由评分	$m \times 7168 \rightarrow m \times 896$	7168×896	$2 \times m \times 7168 \times 896$	92
进入专家潜空间	$m \times 7168 \rightarrow m \times 3584$	7168×3584	$2 \times m \times 7168 \times 3584$	92
路由专家 e : 潜空间变换	$t_e \times 3584 \rightarrow t_e \times 3072 \rightarrow t_e \times 3584$	3584×3072 两个; 3072×3584 一个	$6 \times t_e \times 3584 \times 3072$	92
合并后离开潜空间	$m \times 3584 \rightarrow m \times 7168$	3584×7168	$2 \times m \times 3584 \times 7168$	92
共享专家: 两个专家宽度合并	$m \times 7168 \rightarrow m \times 6144 \rightarrow m \times 7168$	7168×6144 两个; 6144×7168 一个	$6 \times m \times 7168 \times 6144$	92

深度连接

模块及作用	输入 → 输出 (行数 × 宽度) 出宽)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
AttnRes: 深度方向混合	可供选择的块表示 → 当前主干表示	深度混合参数	按每层参与混合的块数计, 不作为上下文 KV	按参与混合的块数累计

输出接口

模块及作用	输入 → 输出 (行数 × 宽度) 出宽)	权重矩阵 (输入宽 × 输出宽)	矩阵 FLOPs	层 / 调用数
词表头 (末端一次)	$m_{\text{out}} \times 7168 \rightarrow m_{\text{out}} \times 163840$	7168×163840	$2 \times m_{\text{out}} \times 7168 \times 163840$	1

累计时, 用各行公式乘最后一列的层数; 路由专家再对实际接收行数 t_e 求和。末端归一化执行一次, 层内归一化、激活、位置处理和状态更新沿相应分支执行。

阅读 Kimi K3 的表时, 需要分别看注意力和前馈网络的层数: 69 层采用 KDA, 24 层采用 MLA; 第一层采用稠密 FFN, 后 92 层采用潜空间 MoE。共享专家保持 7168 维主干输入, 路由专家先经过降维投影, 处理 3584 维输入。表中的层数、宽度与 t_e 因而分别控制重复次数、单次尺寸与实际调用量。³²

本章小结

设计模型时, 也可以先从设备容量中扣除服务所需的状态和工作区, 再比较剩余空间能够容纳的模型规模与结构。MQA、GQA 与稀疏激活说明, 提高系统效率也是选择模型结构的重要依据; 在容量允许的范围内, 还要比较质量、时延与成本, 才能确定设计方案。

本章把第一章的三个资源量展开为可计算的对象。矩阵尺寸给出权重容量与线性运算, 查询—上下文位置对给出注意力交互, 缓存的保存时间与模型调用次数决定状态的存储量和读取量, 专家分派决定每批使用多少份权重。面对新条件, 先指出它改变了哪一个对象, 再重新代入相应公式: 增加 batch、延长上下文、扩大专家集合和增加选中专家数, 会作用于不同的项。

读完本章, 应能从新配置列出一前向、一次 decode 和完整生成的需求, 并解释长度或 batch 改变时哪些项增长。第三章将进一步考虑请求的到达过程与工具调用的依赖关系, 使这些需求成为随时间变化的负载。

³²完整前向记录。